Homework 1: Getting Started

This homework introduces the environment and tools you will be using to complete your future project assignments. It includes a quick C primer. You should use this assignment to familiarize yourself with the tools you will be using throughout the course.

1 Software engineering

Best practices

A good software engineer strives to write programs that are fast, correct, and maintainable. Here are a few best practices which we feel are worth reminding you of:

- Maintainability: comment your code, use meaningful variable names, insert whitespaces, and follow a consistent style.
- Code organization: break up large functions into smaller subroutines, write reusable helper functions, and avoid duplicate code.
- Version control: write descriptive commit messages, and commit your changes frequently (but don't commit anything which doesn't compile).
- Assertions: frequently make assertions within your code so that you know quickly when something goes wrong.

Pair programming

Pair programming is a technique in which two programmers work on the same machine. According to Laurie Williams from North Carolina State University, "One of the programmers, the driver, has control of the keyboard/mouse and actively implements the program. The other programmer, the observer, continuously observes the work of the driver to identify tactical (syntactic, spelling, etc.) defects, and also thinks strategically about the direction of the work." The programmers work equally to develop a piece of software as they periodically switch roles.

You will gain more experience with pair programming during Project 1.

[Note: This course makes use of AWS and Git features which may not be available to all OCW users.]

4 C Primer

This section will be a short introduction to the C programming language. The code used in the exercises is located in `homework1/c-primer` in your repository.

Why use C?

- Simple: No complicated object-oriented abstractions like Java/C++.
- Powerful: Offers direct access to memory (but does not offer protection in accessing memory).
- Fast: No overhead of a runtime or JIT compiler, and no behind-the-scenes runtime features (like garbage collection) that use machine resources.
- Ubiquitous: C is the most popular language for low-level and performance-intensive software like device drivers, operating system kernels, and microcontrollers.

Preprocessing

The C preprocessor modifies source code before it is passed to the compilation phase. Specifically, it performs string substitution of `#define` macros, conditionally omits sections of code, processes `#include` directives to import entire files' worth of code, and strips comments from code.

As an example, consider the code in `preprocess.c`, which is replicated in Figure 1.

```
01 // All occurrences of ONE will be replaced by 1.
02 #define ONE 1
03
04 // Macros can also behave similar to inline functions.
05 // Note that parentheses around parameters are required to preserve order of
06 // operations. Otherwise, you can introduce bugs when substitution happens.
07 #define MIN(a,b) ((a) < (b) ? (a) : (b))
08
09 int c = ONE, d = ONE + 5;
10 int e = MIN(c, d);
11
12 #ifndef NDEBUG
13 // This code will be compiled only when
14 // the macro NDEBUG is not defined.
15 // Recall that if clang is passed -DNDEBUG on the command line,
16 // then NDEBUG will be defined.
17 if (something) {}
18 #endif
```

Figure 1: A sample C program. If -DNDEBUG is not on, the preprocessor will omit the `if` statement in line 17.

Exercise: Direct `clang` to preprocess `preprocess.c`.

```
$ clang -E preprocess.c
```

The preprocessed code will be output to the console. Now rerun the C preprocessor with the following command:

```
$ clang -E -DNDEBUG preprocess.c
```

You will notice that the `if` statement won't appear in the preprocessor output.

Data types and their sizes

C supports a variety of primitive types, including the types listed in Figure 2.

Note: On most 64-bit machines and compilers, a standard-precision value (e.g. `int`, `float`) is 32 bits. A `short` is usually 16 bits, and a `long` or a `double` is usually 64. However, the precisions of these types are weakly defined by the C standard, and may vary across compilers and machines. Confusingly, sometimes `int` and `long` are the same precision, and sometimes `long` and `long long` are the same, both longer than `int`. Sometimes, `int`, `long`, and `long long` all mean the exact same thing!

```c
short s;        // short signed integer
unsigned int i; // standard-length unsigned integer
long l;         // long signed integer
long long l;    // extra-long signed integer
char c;         // represents 1 ASCII character (1 byte)
float f;        // standard-precision floating point number
double d;       // double-precision floating point number
```

Figure 2: Some of the primitive types in C.

For throwaway variables or variables which will stay well under precision limits, use a regular `int`. The precisions of these values are set in order to maximize performance on machines with different word sizes. If you are working with bit-level manipulation, it is better to use unsigned data types such as `uint64_t` (unsigned 64 bit `int`). Otherwise, it is often better to use a non-explicit variable such as a regular `int`.

Furthermore, if you know the architecture you're working with, it is often better to write code with explicit data types instead (such as the ones in Figure 3).

```c
#include <stdint.h>

uint64_t unsigned_64_bit_int;
int16_t signed_16_bit_int;
```

Figure 3: Examples of explicit types in C.

You can define more complex data types by composing primitive types into a `struct`. For example, one example of a `struct` definition in C is provided in Figure 4.

```c
typedef struct {
  int id;
  int year;
} student;

student you;
// access values on a struct with .
you.id = 12345;
you.year = 3;
```

Figure 4: Examples of a `struct` declaration in C.

Exercise: Edit `sizes.c` to print the sizes of each of the following types: `int`, `short`, `long`, `char`, `float`, `double`, `unsigned int`, `long long`, `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`, `uint_fast8_t`, `uint_fast16_t`, `uintmax_t`, `intmax_t`, `__int128`, `int[]` and `student`. Note that `__int128` is a clang

C extension, and not part of standard C. To check the size of an `int` array, print the size of the array `x` declared in the provided code.

To compile and run this code, use the following command:

```
$ make sizes && ./sizes
```

To avoid creating repetitive code, you may find it useful to define a `macro` and call it for each of the types.

If you are interested in learning more about built-in types, check out `http://en.cppreference.com/w/c/types/integer` .

Pointers

Pointers are first-class data types that store addresses into memory. A pointer can store the address of anything in memory, including another pointer. In other words, it is possible to have a pointer to a pointer.

Arrays behave very similarly to pointers: both hold information about the type and location of values in memory. There are a few gotchas involved with treating pointers and arrays equivalently, however.[1]

Consider the following (buggy) snippet of code from `pointer.c` in Figure 5.

[1]For further reading on this, try out the challenge at `https://blogs.oracle.com/ksplice/entry/the_ksplice_pointer_challenge` after class.

```c
int main(int argc, char* argv[]) {   // What is the type of argv?
  int i = 5;
  // The & operator here gets the address of i and stores it into pi
  int* pi = &i;
  // The * operator here dereferences pi and stores the value -- 5 --
  // into j.
  int j = *pi;

  char c[] = "6.172";
  char* pc = c;   // Valid assignment: c acts like a pointer to c[0] here.
  char d = *pc;
  printf("char d = %c\n", d);  // What does this print?

  // compound types are read right to left in C.
  // pcp is a pointer to a pointer to a char, meaning that
  // pcp stores the address of a char pointer.
  char** pcp;
  pcp = argv;  // Why is this assignment valid?

  const char* pcc = c;   // pcc is a pointer to char constant
  char const* pcc2 = c;   // What is the type of pcc2?

  // For each of the following, why is the assignment:
  *pcc = '7';  // invalid?
  pcc = *pcp;  // valid?
  pcc = argv[0];  // valid?

  char* const cp = c;   // cp is a const pointer to char
  // For each of the following, why is the assignment:
  cp = *pcp;  // invalid?
  cp = *argv;  // invalid?
  *cp = '!';  // valid?

  const char* const cpc = c;   // cpc is a const pointer to char const
  // For each of the following, why is the assignment:
  cpc = *pcp;  // invalid?
  cpc = argv[0];  // invalid?
  *cpc = '@';  // invalid?

  return 0;
}
```

Figure 5: An example of valid and invalid pointer usage in C.

Exercise: Compile `pointer.c` using the following command:

```
$ make pointer
```

You will see compilation errors corresponding to the invalid statements mentioned in the above program. Why are these statements invalid? Comment out those invalid statements and recompile the program. (Do not worry if you see additional warnings about unused variables.)

Write-up 2: Answer the questions in the comments in `pointer.c`. For example, why are some of the statements valid and some are not?

Write-up 3: For each of the types in the `sizes.c` exercise above, print the size of a pointer to that type. Recall that obtaining the address of an array or struct requires the & operator. Provide the output of your program (which should include the sizes of both the actual type and a pointer to it) in the writeup.

Argument passing

In C, arguments[2] to a function are passed **by value**. That means that if you pass an integer to function `foo(int f)`, a new variable `f` will be initialized inside `foo` with the same value as the integer you passed in.

For instance, consider the code in Figure 6 that swaps two integers. Why doesn't it work as expected?

```
80 void swap(int i, int j) {
81   int temp = i;
82   i = j;
83   j = temp;
84 }
85
86 int main() {
87   int k = 1;
88   int m = 2;
89   swap(k, m);
90   // What does this print?
91   printf("k = %d, m = %d\n", k, m);
92 }
```

Figure 6: An incorrect implementation of swap in C.

[2]In general, `parameters` are the variables that appear in a function definition, and `arguments` are the data that are actually passed in at runtime.

There are two ways to fix this code. One way is to change `swap()` to be a macro, causing the operations to be evaluated in the scope of the macro invocation. Another way is to change `swap()` to use pointers. We will now ask you to fix the code by using pointers.

Write-up 4: File `swap.c` contains the code to swap two integers. Rewrite the `swap()` function using pointers and make appropriate changes in `main()` function so that the values are swapped with a call to `swap()`. Compile the code with `make swap` and run the program with `./swap`. Provide your edited code in the writeup. Verify that the results of both `sizes.c` and `swap.c` are correct by using the python script `verifier.py`.

5 Basic tools

The code that we will be using in this section is located in `homework1/matrix-multiply`.

Building and running your code

You can build the code by going to the `homework1/matrix-multiply` directory and typing `make`. The program will compile using Tapir, a cutting-edge derivative of Clang/LLVM. Notice that we are only compiling with optimization level 1 (i.e., `-O1`).

Exercise: Modify your Makefile so that the program is compiled using optimization level 3 (i.e., `-O3`).

Write-up 5: Now, what do you see when you type `make clean; make`?

You can then run the built binary by typing `./matrix_multiply`. The program should print out something and then crash with a segmentation fault.

Using a debugger

While running your program, if you encounter a segmentation fault, bus error, or assertion failure, or if you just want to set a breakpoint, you can use the debugging tool GDB.

Exercise: Start a debugging session in GDB:

```
$ gdb --args ./matrix_multiply
```

This command should give you a GDB prompt, at which you should type `run` or `r`:

```
$ (gdb) run
```

Your program will crash, giving you back a prompt, where you can type `backtrace` or `bt` to get a stack trace:

```
Program received signal SIGSEGV, Segmentation fault.
0x0000000000400de4 in matrix_multiply_run ()
(gdb) bt
#0  0x0000000000400de4 in matrix_multiply_run ()
#1  0x0000000000400bbe in main ()
```

This stack trace says that the program crashes in `matrix_multiply_run`, but doesn't tell any other information about the error. In order to get more information, build a "debug" version of the code. First, quit GDB by typing `quit` or `q`:

```
(gdb) q
A debugging session is active.

        Inferior 1 [process 26817] will be killed.

Quit anyway? (y or n) y
```

Next, build a "debug" version of the code by typing `make DEBUG=1`:

```
$ make DEBUG=1
clang -g -DDEBUG -O0 -Wall -std=c99 -D_POSIX_C_SOURCE=200809L   -c -o testbed.o testbed.c
clang -g -DDEBUG -O0 -Wall -std=c99 -D_POSIX_C_SOURCE=200809L   -c -o matrix_multiply.o \
matrix_multiply.c
clang -o matrix_multiply testbed.o matrix_multiply.o -lrt -flto -fuse-ld=gold
```

The major differences from the optimized build are '`-g`' (add debug symbols to your program) and '`-O0`' (compile without any optimizations). Once you have created a debug build, you can start a debugging session again:

```
$ gdb --args ./matrix_multiply
(gdb) r
...
Program received signal SIGSEGV, Segmentation fault.
0x00000000004011cf in matrix_multiply_run (A=0x603270, B=0x6031d0, C=0x603130)
    at matrix_multiply.c:90
90              C->values[i][j] += A->values[i][k] * B->values[k][j];
```

Now, GDB can tell that a segmentation fault occurs at at `matrix_multiply.c` line 90. You can ask GDB to print values using `print` or `p`:

```
gdb) p A->values[i][k]
$1 = 7
(gdb) p B->values[k][j]
Cannot access memory at address 0x0
(gdb) p B->values[k]
$2 = (int *) 0x0
(gdb) p k
$3 = 4
```

This suggests that `B->values[4]` is `0x0`, which means B doesn't have row 5. There is something wrong with the matrix dimensions.

Using assertions

The `tbassert` package is a useful tool for catching bugs before your program goes off into the weeds. If you look at `matrix_multiply.c`, you should see some assertions in `matrix_multiply_run` routine that check that the matrices have compatible dimensions.

Exercise: Uncomment these lines and a line to include `tbassert.h` at the top of the file. Then, build and run the program again using GDB. Make sure that you build using `make DEBUG=1`. You should see:

```
(gdb) r
...
Running matrix_multiply_run()...
matrix_multiply.c:80 (int matrix_multiply_run(const matrix *, const matrix *, matrix *))
    Assertion A->cols == B->rows failed: A->cols = 5, B->rows = 4

Program received signal SIGABRT, Aborted.
0x00007ffff7843c37 in raise () from /lib/x86_64-linux-gnu/libc.so.6
```

Now, GDB tells that "`Assertion 'A->cols == B->rows' failed`", which is much better than the former segmentation fault. The assertion provides a printf-like API that allows you to print values in your own output, as above. However, even if you don't print values in your assertions, the debug build still has the symbols for GDB, as above. Unlike the above, however, if you try to print `A->cols`, you will fail. The reason is that GDB is not in the stack frame you want. You can get the stack trace to see which frame you want (#3 in this case), and type `frame 3` or `f 3` to move to frame #3. After that, you can print `A->cols` and `B->cols`.

```
(gdb) bt
#0  0x00007ffff7843c37 in raise () from /lib/x86_64-linux-gnu/libc.so.6
#1  0x00007ffff7847028 in abort () from /lib/x86_64-linux-gnu/libc.so.6
#2  0x000000000040121b in matrix_multiply_run (A=0x603270, B=0x6031d0, C=0x603130)
    at matrix_multiply.c:79
#3  0x0000000000400db2 in main (argc=1, argv=0x7fffffffdf58) at testbed.c:127
(gdb) f 3
#3  0x0000000000400db2 in main (argc=1, argv=0x7fffffffdf58) at testbed.c:127
127       matrix_multiply_run(A, B, C);
(gdb) p A->cols
$1 = 5
(gdb) p B->rows
$2 = 4
```

You should see the values 5 and 4, which indicates that we are multiplying matrices of incompatible dimensions.

You will also see an assertion failure with a line number for the failing assertion without using GDB. Since the extra checks performed by assertions can be expensive, they are disabled for optimized builds, which are the default in our Makefile. As a result, if you make the program without `DEBUG=1`, you will not see an assertion failure.

You should consider sprinkling assertions throughout your code to check important invariants in your program, since they will make your life easier when debugging. In particular, most nontrivial loops and recursive functions should have an assertion of the loop or recursion invariant.

Exercise: Fix `testbed.c`, which creates the matrices, rebuild your program, and verify that it now works. You should see "Elapsed execution time..." after running

```
$ ./matrix_multiply
```

Commit and push your changes to the Git repository:

```
$ git commit -am 'Your commit message'
```

```
$ git push origin master
```

Next, check the result of the multiplication. Run

```
$ ./matrix_multiply -p
```

The program will print out the result. The result seems to be wrong, however. You can check the multiplication of zero matrices by running

```
$ ./matrix_multiply -pz
```

Using a memory checker

Some memory bugs do not crash the program, so GDB cannot tell you where the bug is. You can use the memory checking tools AddressSanitizer and Valgrind to track these bugs.

AddressSanitizer

AddressSanitizer is a quick memory error checker that uses compiler instrumentation and a run-time library. It can detect a wide variety of bugs (including memory leaks).

To use AddressSanitizer, we need to pass the appropriate flags. First, do

```
$ make clean
```

to get rid of the existing build.

Next, do

```
$ make ASAN=1
```

to build with AddressSanitizer's instrumentation.

```
$ make ASAN=1
clang -O1 -g -fsanitize=address -Wall -std=c99 -D_POSIX_C_SOURCE=200809L -c \
testbed.c -o testbed.o
clang -O1 -g -fsanitize=address -Wall -std=c99 -D_POSIX_C_SOURCE=200809L -c \
 matrix_multiply.c -o matrix_multiply.o
clang -o matrix_multiply testbed.o matrix_multiply.o -lrt -flto -fuse-ld=gold \
-fsanitize=address
```

Finally, run the program with

```
$ ./matrix_multiply
```

Write-up 6: What output do you see from AddressSanitizer regarding the memory bug? Paste it into your writeup here.

Valgrind

Valgrind is another tool for checking memory leaks. If you want to check a program but are not able to instrument it, Valgrind is a good option for detecting memory bugs.

Exercise: First, do

```
$ make clean && make
```

to get rid of the existing build and get a fresh build. Run Valgrind using

```
$ valgrind ./matrix_multiply -p
```

You need the `-p` switch, since Valgrind only detects memory bugs that affect outputs. You should also use a "debug" version to get a good result. This command should print out many lines. The important ones are

```
==43644== Use of uninitialised value of size 8
==43644==    at 0x508899B: _itoa_word (_itoa.c:179)
==43644==    by 0x508C636: vfprintf (vfprintf.c:1660)
==43644==    by 0x50933D8: printf (printf.c:33)
==43644==    by 0x401137: print_matrix (matrix_multiply.c:68)
==43644==    by 0x400E0E: main (testbed.c:133)
```

This output indicates that the program used a value before initializing it. The stack trace indicates that the bug occurs in `testbed.c:133`, which is where the program prints out matrix C.

Exercise: Fix `matrix_multiply.c` to initialize values in matrices before using them. Keep in mind that the matrices are stored in structs. Rebuild your program, and verify that it outputs a correct answer. Again, commit and push your changes to the Git repository.

Write-up 7: After you fix your program, run `./matrix_multiply -p`. Paste the program output showing that the matrix multiplication is working correctly.

Memory management

The C programming language requires you to free memory after you are done using it, or else you will have a memory leak. Valgrind can track memory leaks in the program. Run the same Valgrind command, and you will see these lines at the very end:

```
==2158== LEAK SUMMARY:
==2158==    definitely lost: 48 bytes in 3 blocks
==2158==    indirectly lost: 288 bytes in 15 blocks
==2158==      possibly lost: 0 bytes in 0 blocks
==2158==    still reachable: 0 bytes in 0 blocks
==2158==         suppressed: 0 bytes in 0 blocks
```

This output suggests that there are indeed memory leaks in the program. To get more information, you can build your program in debug mode and again run Valgrind, using the flag `--leak-check=full`

```
$ valgrind --leak-check=full ./matrix_multiply -p
```

The trace shows that all leaks are from the creations of matrices `A`, `B`, and `C`.

Exercise: Fix `testbed.c` by freeing these matrices after use with the function `free_matrix`. Rebuild your program, and verify that Valgrind doesn't complain about anything. Commit and push your changes to the Git repository.

Write-up 8: Paste the output from Valgrind showing that there is no error in your program.

Checking code coverage

Bugs may exist in code that doesn't get executed in your tests. You may find it surprising when someone testing your code (like a professor or a TA) uncovers a crash on a line that you never exercised. Additionally, lines that are frequently executed are good candidates for optimization. The `Gcov` tool provides a line-by-line execution count for your program.

Exercise: To use `Gcov`, modify your Makefile and add the flags `-fprofile-arcs` and `-ftest-coverage` to the CFLAGS and LDFLAGS variables. You will have to rebuild from scratch using `make clean` followed by `make DEBUG=1`. Try running your code normally with `./matrix_multiply -p`. Note that a number of new .gcda and .gcno files were created during your execution.

Now use the llvm-cov commandline utility on `testbed.c`:

```
$ llvm-cov gcov testbed.c
```

A new file, `testbed.c.gcov` was created that is identical to the original `testbed.c`, except that it has the number of times each line was executed in the code. In that file, you will see:

```
164 1:       99:
165 if (use_zero_matrix) {
166 #####:  100:    for (int i = 0; i < A->rows; i++) {
167 #####:  101:        for (int j = 0; j < A->cols; j++) {
168 #####:  102:            A->values[i][j] = 0;
169 #####:  103:        }
170 #####:  104:    }
171 #####:  105:    for (int i = 0; i < B->rows; i++) {
172 #####:  106:      for (int j = 0; j < B->cols; j++) {
173 #####:  107:        B->values[i][j] = 0;
174 #####:  108:      }
175 #####:  109:    }
```

The hash-marks indicate lines that were never executed. In general, it is unusual to run a code-coverage utility on a testbed, but a set of untested lines in your core code could lead to unexpected results when executed by someone else.

Another handy use of Gcov is identifying which lines got executed the *most* frequently. Code that gets run the most is often the most costly in terms of performance. Run `llvm-cov gcov matrix_multiply.c` and look at the output:

```
 10:   93:  for (int i = 0; i < A->rows; i++) {
 40:   94:    for (int j = 0; j < B->cols; j++) {
160:   95:      for (int k = 0; k < A->cols; k++) {
 64:   96:        C->values[i][j] += A->values[i][k] * B->values[k][j];
 64:   97:      }
 16:   98:    }
  4:   99:  }
```

These are the loops in `matrix_multiply_run`. Clearly, this function is a good candidate for optimization.

When you are done using Gcov, remove the flags you added to the Makefile because they add costly overhead to the execution, and will negatively impact your actual performance numbers. You should never run benchmarks on code that is instrumented with Gcov. Don't forget to `make clean` to remove the instrumented object files.

6 Using AWSRUN

You share the `athena.dialup.mit.edu` machines with all of MIT. Do not perform computationally intensive tasks on these machines. Directly running and timing the execution on your own Amazon VMs may give inaccurate results; you may be running a small VM, and there might also be measurement errors due to interference with your editor or other programs you are running. To get an accurate timing measure on a dedicated machine, you can use the AWSRUN utilities.

When you are ready to do a performance run you can use

```
$ awsrun ./matrix_multiply
```

and your job will be queued. The command will not return until the job has been completed and you get results unless you control-C twice (canceling the process). Once the job is complete, your output should look something like below:

```
$ awsrun ./matrix_multiply

Submitting Job: ./matrix_multiply
Waiting for job to finish...
==== Standard Error ====
Setup
Running matrix_multiply_run()...



==== Standard Output ====
Elapsed execution time: 0.000001 sec
```

Performance enhancements

To get an idea of the extent to which performance optimizations can affect the performance of your program, we will first increase the size of the input to demonstrate the effects of changes in the code.

Exercise: Increase the size of all matrices to 1000×1000.

Now let's try one of the techniques from the first lecture. Right now, the inner loop produces a sequential access pattern on `A` and skips through memory on `B`.

Let's rearrange the loops to produce a better access pattern.

Exercise: First, you should run the program as is to get a performance measurement. Next, swap the `j` and `k` loops, so that the inner loop strides sequentially through the rows of the `C` and `B` matrices. Rerun the program, and verify that you have produced a speedup. Commit and push your changes to the Git repository.

> **Write-up 9:** Report the execution time of your programs before and after the optimization.

Compiler optimizations

To get an idea of the extent to which compiler optimizations can affect the performance of your program, rebuild your program in "debug" mode and run it with AWSRUN.

Exercise: Rebuild it again with optimizations (just type `make`), and run it with AWSRUN. Both versions should print timing information, and you should verify that the optimized version is faster.

> **Write-up 10:** Report the execution time of your programs compiled in debug mode with `-O0` and in non-debug mode with `-O3`.

7 C style guidelines

Code that adheres to a consistent style is easier to read and debug. Google provides a style guide for C++ which you may find useful: `https://google.github.io/styleguide/cppguide.html`

We have provided a Python script `clint.py`, which is designed to check a subset of Google's style guidelines for C code. To run this script on all source files in your current directory use the command:

```
$ python clint.py *
```

The code the staff provides has no style errors. We suggest, but do not require, that you use this tool to clean up your source code. Part of your code-quality grade on projects is based on the readability of your code. It can be difficult to maintain a consistent style when multiple people are working on the same codebase. For this reason, you may find it useful to use the style checkers provided by the 6.172 staff during your group projects to keep your code readable.

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 3

Homework 2: Profiling Serial Merge Sort

[Note: This assignment makes use of AWS and/or Git features which may not be available to OCW users.]

In this homework you will be introduced to important profiling tools. You'll learn to use Perf, which gives you details about where time is being spent in your program, and Cachegrind, a cache and branch-prediction profiler in the Valgrind tool suite. These tools will help you identify parts of your code that would benefit from further optimization. Finally, you'll use these tools to optimize a serial merge sort routine.

Generally, when you are concerned about the performance of a program, the best approach is to implement something correct and then evaluate it. In some cases, many parts of this initial implementation (or even the entire thing) may be adequate for your needs. However, when you need to improve performance, you must first decide where to focus your efforts. This is where profiling becomes useful. Profiling can help you identify the performance bottlenecks in your program.

1 Getting Started

Code structure

The code for the recitation is in the `recitation` directory. You'll be looking at the files `isort.c`, which contains code for an insertion sorting routine, and `sum.c`, which allocates a large array of integers and sums a sample of them. For the homework, the program you are responsible for improving is in the `homework` directory.

Building the code

In general, `make <target>` will build the code for a specific target binary. Remember that to build with debugging symbols and assertions (useful for debugging with `gdb`), you must build with

```
$ make <target> DEBUG=1
```

Whenever a question asks about performance, we want you to run your programs on the AWS job machines using awsrun.

Submitting your solutions

Remember to explicitly add new files to your repository before committing and pushing your final changes.

```
$ git add
$ git commit -a
$ git status
$ git push
```

If `git status` shows any modified files, then you probably haven't checked your code into your repository properly. We recommend you commit often.

2 Recitation: Perf and Cachegrind

First, we will explore how to use two extremely useful performance profiling tools: Perf and Cachegrind. These tools allow you to identify the performance bottlenecks in a program and measure salient performance properties, such as cache and branch misses. Used correctly, they can help you figure out exactly why your code is running slower than it should.

2.1 Perf

`perf` is a profiler tool for Linux 2.6+ based systems. It uses sampling to gather data about important software, kernel, and hardware events in order to locate performance bottlenecks in a program. It generates a detailed record of where the time is spent in your code.

Note: If `perf` is not installed on your AWS VM, run

```
$ sudo apt install linux-tools-common linux-tools-aws
```

You can use `perf record` and `perf report` to analyze the performance of your program. `perf record` generates a record of the events that occur when you run your code, and `perf report` allows you to view it interactively.

Note: For `perf` to annotate your code properly, you must pass the `-g` flag to `clang` when compiling your program. This flag, which does not affect performance, tells the compiler to generate `debugging symbols`, which allow tools like `perf` and `gdb` to associate lines of machine code with lines of source code. The provided Makefile will do this if you run it with DEBUG=1.

To record performance events on a program, run the following:

```
$ perf record <program_name> <program_arguments>
```

You can then view the results by running `perf report` from the same directory:

```
$ perf report
```

Note: You may see a warning screen that says something like "Kernel address maps were restricted." This is OK, just press any key to continue to the report.

The personal AWS machines are generally less performant (and less consistent) than the dedicated AWSRUN machines, which are accessed via `awsrun`. You can also profile performance in the cloud:

```
$ awsrun perf record <program_name> <program_arguments>
```

This will generate a report on an AWS cloud queue machine rather than on your personal instance, so you need to pass some extra arguments to `perf report` in order to view it. We provide the command `aws-perf-report` to do this for you:

```
$ aws-perf-report
```

Note: `aws-perf-report` should be run **without** `awsrun`.

Spend some time exploring the contents of the report, and try to figure out what it means.

You can see both the C code and the assembly instructions in the annotated output. The performance counter events are usually associated with the instruction right after or a few instructions below the one that causes extra stalls.

File `isort.c` contains an insertion sort routine. Compile the program with

```
$ make isort DEBUG=1
```

Now, running `./isort n k` will sort an array of `n` elements `k` times. Run:

```
$ awsrun perf record ./isort 10000 10
$ aws-perf-report
```

Identify branch misses, clock cycles and instructions. Diagnose the performance bottlenecks in the program.

Checkoff Item 1: Make note of one bottleneck.

2.2 Cachegrind

Cachegrind (a Valgrind tool) is a cache and branch-prediction profiler. Recall from class that a read from the L1 cache can be around 100x faster than a read from RAM! Optimizing for cache hits is a critical part of perfomance engineering.

On virtual environments like those on AWS, hardware events providing information about branches and cache misses are often unavailable, so `perf` may not be helpful. Cachegrind simulates how your program interacts with a machine's cache hierarchy and branch predictor and can be used even in the absence of available hardware performance counters.

Here is an example on how to identify cache misses, branch misses, clock cycles, and instructions executed by your program using Cachegrind:

```
$ valgrind --tool=cachegrind --branch-sim=yes <program_name> <program_arguments>
```

Note: Although `valgrind --tool=cachegrind` measures cache and branch predictor behavior using a simulator, it bases its simulation upon the architecture on which it is run. You should expect different results when running on different machines, e.g. when running on your personal instance vs. the awsrun machines.

File `sum.c` contains a program that allocates an array of $U = 10$ million elements, and then sums $N = 100$ million elements chosen at random. Make it with:

```
$ make sum
```

Checkoff Item 2: Run `sum` under `cachegrind` to identify cache performance. It may take a little while. In the output, look at the D1 and LLd misses. D1 represents the lowest-level cache (L1), and LL represents the last (highest) level data cache (on most machines, L3). Do these numbers correspond with what you would expect? Try playing around with the values N and U in sum.c. How can you bring down the number of cache misses?

Hint: to find information about your CPU and its caches, use `lscpu`.

Checkoff: Explain to a TA your responses to the previous two Checkoff Items.

3 Homework: Sorting

Profiling code can often give you valuable insight into how a program works and why it performs the way it does. In this exercise, we have provided a simple implementation of merge sort in `homework/sort_a.c`. You can make the code for all sort implementations by just typing `make`. After making, run the code with `./sort <num_elements> <num_trials>`.

Write-up 1: Compare the Cachegrind output on the `DEBUG=1` code versus `DEBUG=0` compiler optimized code. Explain the advantages and disadvantages of using instruction count as a substitute for time when you compare the performance of different versions of this program.

Make sure you're evaluating the non-debug version for the rest of your experiments.

We want to identify performance bottlenecks and incrementally improve the performance of the merge sort as explained in the tasks below. As you make improvements to the sort routine in each task, we would like you to keep a copy of the sort routine before the improvements and keep adding new versions of the sort routine in the testing suite (with different names), so we can profile and compare the performance of different versions. The testing code is set up so that you can easily add new sorting routines to test, **as long as you keep the same function signature for**

the sort routine, i.e., each sort routine you make should have the same type as `sort_a` provided in `sort_a.c`. In addition, **do not remove the `static` keyword in any of the internal functions, preserve the structure of the merge sort algorithm as coded, and do not change it radically for this homework**.

3.1 Inlining

You would like to see how much inline functions can help. Copy over the code from `sort_a.c` into `sort_i.c`, and change all the routine names from `<function>_a` to `<function>_i`. Using the `inline` keyword, inline one or more of the functions in `sort_i.c` and `util.c`. To add the `sort_i` routine to the testing suite, uncomment the line in `main.c`, under `testFunc`, that specifies `sort_i`. Profile and annotate the inlined program.

Write-up 2: Explain which functions you chose to inline and report the performance differences you observed between the inlined and uninlined sorting routines.

The compiler does not always inline functions with the `inline` keyword. Use the annotated assembly code to help you verify whether the compiler really inlined the functions you wanted to inline.

Try to inline the `sort_i` recursive function (You may find forward declarations useful). When you inline a recursive function, it will expand the recursion only a fixed number of times, and then will recurse.

Write-up 3: Explain the possible performance downsides of inlining recursive functions. How could profiling data gathered using cachegrind help you measure these negative performance effects?

3.2 Pointers vs Arrays

You learn that pointer access can be more efficient than array indexing when manipulating arrays, and you want to implement this efficiency in the sorting algorithm, in addition to the inlining changes you did in the previous task. To accomplish this, you first copy your sort routine in `sort_i.c` into `sort_p.c` and update the function names in `sort_p.c` to `<function>_p`. Then you modify the code in `sort_p.c` to use pointers instead of arrays and include `sort_p` into the test suite.

Write-up 4: Give a reason why using pointers may improve performance. Report on any performance differences you observed in your implementation.

3.3 Coarsening

The base case in the recursion of `sort_p` routine in `sort_p.c` is sorting just one element. You want to coarsen the recursion so that the recursion base case sorts more than one element. As before, copy all your changes into `sort_c.c`, and update your function names. Use the fastest correct sort routine so far as a base for `sort_c.c`. Then coarsen the recursion in `sort_c.c`. We have provided an insertion sort routine in `isort.c`, which you can use as the sorting algorithm in your base case. You can also write your favorite sorting algorithm and use it for your base case. Remember that in order to use a function defined elsewhere, you first need to write a function prototype. Include `sort_c` into your test suite.

Write-up 5: Explain what sorting algorithm you used and how you chose the number of elements to be sorted in the base case. Report on the performance differences you observed.

3.4 Reducing Memory Usage

Observe that two temporary memory scratch spaces `left` and `right` are used in the `merge_c` function. You want to optimize memory by using just one temporary memory scratch space (thereby reducing the total temporary memory usage by half) and using the input array to be sorted as the other memory scratch space in merge operation. As before, copy your newest changes into `sort_m.c` and update the function names. Then implement the memory optimization described above.

Write-up 6: Explain any difference in performance in your `sort_m.c`. Can a compiler automatically make this optimization for you and save you all the effort? Why or why not?

3.5 Reusing Temporary Memory

Though you have reduced temporary memory usage in `sort_m.c` by half, you find that it is unnecessary to allocate and deallocate the temporary memory scratch space in the `merge_m` function every time it is called. Instead, you would like to allocate the required memory once in the beginning and deallocate it at the end after sorting is done. To do this final optimization, copy your

sorting routine into `sort_f.c`. Then, implement the memory enhancement described above in `sort_f.c` and include `sort_f` in your test suite.

Write-up 7: Report any differences in performance in your `sort_f.c`, and explain the differences using profiling data.

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 5

Homework 3: Vectorization

In this homework and recitation you will experiment with Intel Vector Extensions. You will learn how to vectorize your code, figure out when vectorization has succeeded and debug when vectorization seems to have worked but you aren't seeing speedup.

Vectorization is a general optimization technique that can buy you an order of magnitude performance increase in some cases. It is also a delicate operation. On the one hand, vectorization is automatic: when `clang` *is told to optimize aggressively, it will automatically try to vectorize every loop in your program. On the other hand, very small changes to loop structure cause* `clang` *to give up and not vectorize at all. Furthermore, these small changes may allow your code to vectorize but not yield the expected speedup. We will discuss how to identify these cases so that you can get the most out of your vector units.*

1 Getting started

[Note: This assignment makes use of AWS and/or Git features which may not be available to OCW users.]

Submitting your solutions

For each question we ask (i.e., each sentence with a question mark), respond with a short (1-3 sentence) responses or a code snippet (if requested). **Please ensure that all the times you quote are obtained from the `awsrun` machines.**

2 Vectorization in `clang`

Consider a loop that performs elementwise addition between two arrays `A` and `B`, storing the result in array `C`. This loop is *data parallel* because the operation during any iteration i_1 is independent of the operation during any iteration i_2 where $i_1 \neq i_2$. In short, the compiler should be

allowed to schedule each iteration in any order, or pack multiple iterations into a single clock cycle. The first option will be covered in the next homework. The second case is covered by vectorization, also known as "single instruction, multiple data" or SIMD.

Vectorization is a delicate operation: very small changes to loop structure may cause `clang` to give up and not vectorize at all, or to vectorize your code but not yield the expected speedup. Occasionally, unvectorized code may be faster than vectorized code. Before we can understand this fragility, we must get a handle on how to interpret what `clang` is actually doing when it vectorizes code; in Section 3, you will see the actual performance impacts of vectorizing code.

2.1 Example 1

We will start with the following simple loop:

```
01 #include <stdint.h>
02 #include <stdlib.h>
03 #include <math.h>
04
05 #define SIZE (1L << 16)
06
07 void test(uint8_t * a,  uint8_t * b) {
08   uint64_t i;
09
10   for (i = 0; i < SIZE; i++) {
11     a[i] += b[i];
12   }
13 }
```

```
$ make clean; make ASSEMBLE=1 VECTORIZE=1 example1.o
```

You should see the following output, informing you that the loop has been vectorized. Although `clang` does tell you this, you should always look at the assembly to see exactly how it has been vectorized, since it is not guaranteed to be using the vector registers optimally.

```
14 example1.c:12:3: remark: vectorized loop (vectorization width: 16, interleaved count: 2)
15       [-Rpass=loop-vectorize]
16   for (i = 0; i < SIZE; i++) {
```

Now, let's inspect the assembly code in `example1.s`. You should see something similar to the following:

```
# %bb.0:                                # %entry
        #DEBUG_VALUE: test:a <- %rdi
        #DEBUG_VALUE: test:a <- %rdi
        #DEBUG_VALUE: test:b <- %rsi
        #DEBUG_VALUE: test:b <- %rsi
        #DEBUG_VALUE: test:i <- 0
        .loc        1 12 3 prologue_end     # example1.c:12:3

        leaq        65536(%rsi), %rax
        cmpq        %rdi, %rax
        jbe        .LBB0_2
# %bb.1:                                # %entry
        #DEBUG_VALUE: test:b <- %rsi
        #DEBUG_VALUE: test:a <- %rdi
        leaq        65536(%rdi), %rax
        cmpq        %rsi, %rax
        jbe        .LBB0_2
# %bb.4:                                # %for.body.preheader
        #DEBUG_VALUE: test:b <- %rsi
        #DEBUG_VALUE: test:a <- %rdi
        .loc        1 0 3 is_stmt 0         # example1.c:0:3

        movq        $-65536, %rax           # imm = 0xFFFF0000

        .p2align        4, 0x90
.LBB0_5:                                # %for.body
                                        # =>This Inner Loop Header: Depth=1
        #DEBUG_VALUE: test:b <- %rsi
        #DEBUG_VALUE: test:a <- %rdi
.Ltmp0:
        .loc        1 13 13 is_stmt 1       # example1.c:13:13

        movzbl        65536(%rsi,%rax), %ecx
        .loc        1 13 10 is_stmt 0       # example1.c:13:10

        addb        %cl, 65536(%rdi,%rax)
        .loc        1 13 13                 # example1.c:13:13

        movzbl        65537(%rsi,%rax), %ecx
        .loc        1 13 10                 # example1.c:13:10

        addb        %cl, 65537(%rdi,%rax)
        .loc        1 13 13                 # example1.c:13:13

        movzbl        65538(%rsi,%rax), %ecx
        .loc        1 13 10                 # example1.c:13:10

        addb        %cl, 65538(%rdi,%rax)
        .loc        1 13 13                 # example1.c:13:13

        movzbl        65539(%rsi,%rax), %ecx
        .loc        1 13 10                 # example1.c:13:10

        addb        %cl, 65539(%rdi,%rax)
```

```
.Ltmp1:
    .loc    1 12 17 is_stmt 1       # example1.c:12:17

    addq    $4, %rax
.Ltmp2:
    .loc    1 12 3 is_stmt 0        # example1.c:12:3

    jne .LBB0_5
    jmp .LBB0_6
.LBB0_2:                                # %vector.body.preheader
    #DEBUG_VALUE: test:b <- %rsi
    #DEBUG_VALUE: test:a <- %rdi
    .loc    1 0 3                   # example1.c:0:3

    movq    $-65536, %rax           # imm = 0xFFFF0000
    .p2align    4, 0x90
.LBB0_3:                                # %vector.body
                                        # =>This Inner Loop Header: Depth=1
    #DEBUG_VALUE: test:b <- %rsi
    #DEBUG_VALUE: test:a <- %rdi
.Ltmp3:
    .loc    1 13 13 is_stmt 1       # example1.c:13:13

    movdqu  65536(%rsi,%rax), %xmm0
    movdqu  65552(%rsi,%rax), %xmm1
    .loc    1 13 10 is_stmt 0       # example1.c:13:10

    movdqu  65536(%rdi,%rax), %xmm2
    paddb   %xmm0, %xmm2
    movdqu  65552(%rdi,%rax), %xmm0
    movdqu  65568(%rdi,%rax), %xmm3
    movdqu  65584(%rdi,%rax), %xmm4
    movdqu  %xmm2, 65536(%rdi,%rax)
    paddb   %xmm1, %xmm0
    movdqu  %xmm0, 65552(%rdi,%rax)
    .loc    1 13 13                 # example1.c:13:13

    movdqu  65568(%rsi,%rax), %xmm0
    .loc    1 13 10                 # example1.c:13:10

    paddb   %xmm3, %xmm0
    .loc    1 13 13                 # example1.c:13:13

    movdqu  65584(%rsi,%rax), %xmm1
    .loc    1 13 10                 # example1.c:13:10

    movdqu  %xmm0, 65568(%rdi,%rax)
    paddb   %xmm4, %xmm1
    movdqu  %xmm1, 65584(%rdi,%rax)
.Ltmp4:
    .loc    1 12 26 is_stmt 1       # example1.c:12:26

    addq    $64, %rax
    jne .LBB0_3
```

Write-up 1: Look at the assembly code above. The compiler has translated the code to set the start index at -2^{16} and adds to it for each memory access. Why doesn't it set the start index to 0 and use small positive offsets?

This code first checks if there is a partial overlap between array `a` and `b`. If there is an overlap, then it does a simple non-vectorized code. If there is overlap, then go to `.LBB0_2`, and do a vectorized version. The above can, at best, be called partially vectorized. The problem is that the compiler is constrained by what we tell it about the arrays. If we tell it more, then perhaps it can do more optimization. The most obvious thing is to inform the compiler that no overlap is possible. This is done in standard C by using the `restrict` qualifier for the pointers.

```
125 void test(uint8_t * restrict a,  uint8_t * restrict b) {
126   uint64_t i;
127
128   for (i = 0; i < SIZE; i++) {
129     a[i] += b[i];
130   }
131 }
```

Now you should see the following assembly code:

```
132 # %bb.0:                                # %entry
133         #DEBUG_VALUE: test:a <- %rdi
134         #DEBUG_VALUE: test:a <- %rdi
135         #DEBUG_VALUE: test:b <- %rsi
136         #DEBUG_VALUE: test:b <- %rsi
137         movq        $-65536, %rax           # imm = 0xFFFF0000
138 .Ltmp0:
139         #DEBUG_VALUE: test:i <- 0
140         .p2align        4, 0x90
141 .LBB0_1:                                # %vector.body
142                                         # =>This Inner Loop Header: Depth=1
143         #DEBUG_VALUE: test:b <- %rsi
144         #DEBUG_VALUE: test:a <- %rdi
145         .loc        1 13 13 prologue_end    # example1.c:13:13
146
147         movdqu        65536(%rsi,%rax), %xmm0
148         .loc        1 13 10 is_stmt 0       # example1.c:13:10
149
150         movdqu        65536(%rdi,%rax), %xmm1
151         paddb        %xmm0, %xmm1
152         movdqu        65552(%rdi,%rax), %xmm0
153         movdqu        65568(%rdi,%rax), %xmm2
154         movdqu        65584(%rdi,%rax), %xmm3
155         movdqu        %xmm1, 65536(%rdi,%rax)
156         .loc        1 13 13                 # example1.c:13:13
157
158         movdqu        65552(%rsi,%rax), %xmm1
159         .loc        1 13 10                 # example1.c:13:10
160
161         paddb        %xmm1, %xmm0
162         movdqu        %xmm0, 65552(%rdi,%rax)
163         .loc        1 13 13                 # example1.c:13:13
164
165         movdqu        65568(%rsi,%rax), %xmm0
166         .loc        1 13 10                 # example1.c:13:10
167
168         paddb        %xmm2, %xmm0
169         .loc        1 13 13                 # example1.c:13:13
170
171         movdqu        65584(%rsi,%rax), %xmm1
172         .loc        1 13 10                 # example1.c:13:10
173
174         movdqu        %xmm0, 65568(%rdi,%rax)
175         paddb        %xmm3, %xmm1
176         movdqu        %xmm1, 65584(%rdi,%rax)
177 .Ltmp1:
178         .loc        1 12 26 is_stmt 1       # example1.c:12:26
179
180         addq        $64, %rax
181          jne        .LBB0_1
```

The generated code is better, but it is assuming the data are NOT 16 bytes aligned (`movdqu` is unaligned move). It also means that the loop above can not assume that both arrays are aligned. If `clang` were smart, it could test for the cases where the arrays are either both aligned, or both unaligned, and have a fast inner loop. However, it does not do that currently.

So in order to get the performance we are looking for, we need to tell `clang` that the arrays are aligned. There are a couple of ways to do that. The first is to construct a (non-portable) aligned type, and use that in the function interface. The second is to add an intrinsic or two within the function itself. The second option is easier to implement on older code bases, as other functions calling the one to be vectorized do not have to be modified. The intrinsic has for this is called `__builtin_assume_aligned`:

```
182 void test(uint8_t * restrict a,  uint8_t * restrict b) {
183   uint64_t i;
184
185   a = __builtin_assume_aligned(a, 16);
186   b = __builtin_assume_aligned(b, 16);
187
188   for (i = 0; i < SIZE; i++) {
189     a[i] += b[i];
190   }
191 }
```

After you add the instruction `__builtin_assume_aligned`, you should see something similar to the following output:

```
192 # %bb.0:                                # %entry
193         #DEBUG_VALUE: test:a <- %rdi
194         #DEBUG_VALUE: test:a <- %rdi
195         #DEBUG_VALUE: test:b <- %rsi
196         #DEBUG_VALUE: test:b <- %rsi
197         movq        $-65536, %rax           # imm = 0xFFFF0000
198 .Ltmp0:
199         #DEBUG_VALUE: test:i <- 0
200         .p2align        4, 0x90
201 .LBB0_1:                                # %vector.body
202                                         # =>This Inner Loop Header: Depth=1
203         #DEBUG_VALUE: test:b <- %rsi
204         #DEBUG_VALUE: test:a <- %rdi
205         .loc        1 16 10 prologue_end    # example1.c:16:10
206
207         movdqa        65536(%rdi,%rax), %xmm0
208         movdqa        65552(%rdi,%rax), %xmm1
209         movdqa        65568(%rdi,%rax), %xmm2
210         movdqa        65584(%rdi,%rax), %xmm3
211         paddb         65536(%rsi,%rax), %xmm0
212         paddb         65552(%rsi,%rax), %xmm1
213         movdqa        %xmm0, 65536(%rdi,%rax)
214         movdqa        %xmm1, 65552(%rdi,%rax)
215         paddb         65568(%rsi,%rax), %xmm2
216         paddb         65584(%rsi,%rax), %xmm3
217         movdqa        %xmm2, 65568(%rdi,%rax)
218         movdqa        %xmm3, 65584(%rdi,%rax)
219 .Ltmp1:
220         .loc        1 15 26                 # example1.c:15:26
221
222         addq        $64, %rax
223         jne        .LBB0_1
224 .Ltmp2:
```

Now finally, we get the nice tight vectorized code (`movdqa` is aligned move) we were looking for, because `clang` has used packed SSE instructions to add 16 bytes at a time. It also manages to load and store two at a time, which it did not do last time. The question is now that we understand what we need to tell the compiler, how much more complex can the loop be before auto-vectorization fails.

Next, we try to turn on AVX2 instructions using the following command:

```
$ make clean; make ASSEMBLE=1 VECTORIZE=1 AVX2=1 example1.o
```

```
225 # %bb.0:                                # %entry
226         #DEBUG_VALUE: test:a <- %rdi
227         #DEBUG_VALUE: test:a <- %rdi
228         #DEBUG_VALUE: test:b <- %rsi
229         #DEBUG_VALUE: test:b <- %rsi
230         movq        $-65536, %rax           # imm = 0xFFFF0000
231 .Ltmp0:
232         #DEBUG_VALUE: test:i <- 0
233         .p2align        4, 0x90
234 .LBB0_1:                                # %vector.body
235                                         # =>This Inner Loop Header: Depth=1
236         #DEBUG_VALUE: test:b <- %rsi
237         #DEBUG_VALUE: test:a <- %rdi
238         .loc        1 16 10 prologue_end    # example1.c:16:10
239
240         vmovdqu         65536(%rdi,%rax), %ymm0
241         vmovdqu         65568(%rdi,%rax), %ymm1
242         vmovdqu         65600(%rdi,%rax), %ymm2
243         vmovdqu         65632(%rdi,%rax), %ymm3
244         vpaddb          65536(%rsi,%rax), %ymm0, %ymm0
245         vpaddb          65568(%rsi,%rax), %ymm1, %ymm1
246         vpaddb          65600(%rsi,%rax), %ymm2, %ymm2
247         vmovdqu         %ymm0, 65536(%rdi,%rax)
248         vmovdqu         %ymm1, 65568(%rdi,%rax)
249         vmovdqu         %ymm2, 65600(%rdi,%rax)
250         vpaddb          65632(%rsi,%rax), %ymm3, %ymm0
251         vmovdqu         %ymm0, 65632(%rdi,%rax)
252 .Ltmp1:
253         .loc        1 15 26                 # example1.c:15:26
254
255         addq        $128, %rax
256         jne        .LBB0_1
257 .Ltmp2:
```

Write-up 2: This code is still not aligned when using AVX2 registers. Fix the code to make sure it uses aligned moves for the best performance.

2.2 Example 2

Take a look at the second example below in `example2.c`:

```
258 void test(uint8_t * restrict a, uint8_t * restrict b) {
259   uint64_t i;
260
261   uint8_t * x = __builtin_assume_aligned(a, 16);
262   uint8_t * y = __builtin_assume_aligned(b, 16);
263
264   for (i = 0; i < SIZE; i++) {
265     /* max() */
266     if (y[i] > x[i]) x[i] = y[i];
267   }
268 }
```

Compile example 2 with the following command:

```
$ make clean; make ASSEMBLE=1 VECTORIZE=1 example2.o
```

Note that the assembly does not vectorize nicely. Now, change the function to look like the following:

```
269 void test(uint8_t * restrict a, uint8_t * restrict b) {
270   uint64_t i;
271
272   a = __builtin_assume_aligned(a, 16);
273   b = __builtin_assume_aligned(b, 16);
274
275   for (i = 0; i < SIZE; i++) {
276     /* max() */
277     a[i] = (b[i] > a[i]) ? b[i] : a[i];
278   }
279 }
```

Now, you actually see the vectorized assembly with the `movdqa` and `pmaxub` instructions.

```
# %bb.0:                                # %entry
        #DEBUG_VALUE: test:a <- %rdi
        #DEBUG_VALUE: test:a <- %rdi
        #DEBUG_VALUE: test:b <- %rsi
        #DEBUG_VALUE: test:b <- %rsi
        movq        $-65536, %rax           # imm = 0xFFFF0000
.Ltmp0:
        #DEBUG_VALUE: test:i <- 0
        .p2align        4, 0x90
.LBB0_1:                                # %vector.body
                                        # =>This Inner Loop Header: Depth=1
        #DEBUG_VALUE: test:b <- %rsi
        #DEBUG_VALUE: test:a <- %rdi
        .loc        1 17 15 prologue_end    # example2.c:17:15

        movdqa        65536(%rsi,%rax), %xmm0
        movdqa        65552(%rsi,%rax), %xmm1
        .loc        1 17 14 is_stmt 0       # example2.c:17:14

        pmaxub        65536(%rdi,%rax), %xmm0
        pmaxub        65552(%rdi,%rax), %xmm1
        .loc        1 17 12                 # example2.c:17:12

        movdqa        %xmm0, 65536(%rdi,%rax)
        movdqa        %xmm1, 65552(%rdi,%rax)
        .loc        1 17 15                 # example2.c:17:15

        movdqa        65568(%rsi,%rax), %xmm0
        movdqa        65584(%rsi,%rax), %xmm1
        .loc        1 17 14                 # example2.c:17:14

        pmaxub        65568(%rdi,%rax), %xmm0
        pmaxub        65584(%rdi,%rax), %xmm1
        .loc        1 17 12                 # example2.c:17:12

        movdqa        %xmm0, 65568(%rdi,%rax)
        movdqa        %xmm1, 65584(%rdi,%rax)
.Ltmp1:
        .loc        1 15 28 is_stmt 1       # example2.c:15:28

        addq        $64, %rax
        jne        .LBB0_1
.Ltmp2:
```

Write-up 3: Provide a theory for why the compiler is generating dramatically different assembly.

2.3 Example 3

Open up `example3.c` and run the following command:

```
$ make clean; make ASSEMBLE=1 VECTORIZE=1 example3.o
```

```
323 void test(uint8_t * restrict a, uint8_t * restrict b) {
324   uint64_t i;
325
326   for (i = 0; i < SIZE; i++) {
327     a[i] = b[i + 1];
328   }
329 }
```

Write-up 4: Inspect the assembly and determine why the assembly does not include instructions with vector registers. Do you think it would be faster if it did vectorize? Explain.

2.4 Example 4

Take a look at `example4.c`.

```
330 double test(double * restrict a) {
331   size_t i;
332
333   double *x = __builtin_assume_aligned(a, 16);
334
335   double y = 0;
336
337   for (i = 0; i < SIZE; i++) {
338     y += x[i];
339   }
340   return y;
341 }
```

```
$ make clean; make ASSEMBLE=1 VECTORIZE=1 example4.o
```

You should see the non-vectorized code with the `addsd` instruction.

```
342 .LBB0_1:                                # %for.body
343                                         # =>This Inner Loop Header: Depth=1
344         #DEBUG_VALUE: test:x <- %rdi
345         #DEBUG_VALUE: test:a <- %rdi
346 .Ltmp1:
347         #DEBUG_VALUE: test:y <- %xmm0
348         .loc        1 18 7 prologue_end     # example4.c:18:7
349
350         addsd       524288(%rdi,%rax,8), %xmm0
351 .Ltmp2:
352         #DEBUG_VALUE: test:y <- %xmm0
353         addsd       524296(%rdi,%rax,8), %xmm0
354 .Ltmp3:
355         #DEBUG_VALUE: test:y <- %xmm0
356         addsd       524304(%rdi,%rax,8), %xmm0
357 .Ltmp4:
358         #DEBUG_VALUE: test:y <- %xmm0
359         addsd       524312(%rdi,%rax,8), %xmm0
360 .Ltmp5:
361         #DEBUG_VALUE: test:y <- %xmm0
362         addsd       524320(%rdi,%rax,8), %xmm0
363 .Ltmp6:
364         #DEBUG_VALUE: test:y <- %xmm0
365         addsd       524328(%rdi,%rax,8), %xmm0
366 .Ltmp7:
367         #DEBUG_VALUE: test:y <- %xmm0
368         addsd       524336(%rdi,%rax,8), %xmm0
369 .Ltmp8:
370         #DEBUG_VALUE: test:y <- %xmm0
371         addsd       524344(%rdi,%rax,8), %xmm0
372 .Ltmp9:
373         #DEBUG_VALUE: test:y <- %xmm0
374         .loc        1 17 17                 # example4.c:17:17
375
376         addq        $8, %rax
377 .Ltmp10:
378         .loc        1 17 3 is_stmt 0        # example4.c:17:3
379
380         jne        .LBB0_1
```

Notice that this does not actually vectorize as the `xmm` registers are operating on 8 byte chunks. The problem here is that `clang` is not allowed to re-order the operations we give it. Even though the the addition operation is associative with real numbers, they are not with floating point numbers. (Consider what happens with signed zeros, for example.)

Furthermore, we need to tell `clang` that reordering operations is okay with us. To do this, we need to add another compile-time flag, `-ffast-math`. Add the compilation flag `-ffast-math` to the Makefile and compile the program again.

Write-up 5: Check the assembly and verify that it does in fact vectorize properly. Also what do you notice when you run the command

```
$ clang -O3 example4.c -o example4; ./example4
```

with and without the `-ffast-math` flag? Specifically, why do you a see a difference in the output.

```
# %bb.0:                                # %entry
        #DEBUG_VALUE: test:a <- %rdi
        #DEBUG_VALUE: test:a <- %rdi
        #DEBUG_VALUE: test:x <- %rdi
        #DEBUG_VALUE: test:x <- %rdi
        xorpd   %xmm0, %xmm0
.Ltmp0:
        #DEBUG_VALUE: test:i <- 0
        #DEBUG_VALUE: test:y <- 0.000000e+00
        movq    $-65536, %rax           # imm = 0xFFFF0000
        xorpd   %xmm1, %xmm1
        .p2align        4, 0x90
.LBB0_1:                                # %vector.body
                                        # =>This Inner Loop Header: Depth=1
        #DEBUG_VALUE: test:x <- %rdi
        #DEBUG_VALUE: test:a <- %rdi
.Ltmp1:
        .loc    1 18 7 prologue_end     # example4.c:18:7

        addpd   524288(%rdi,%rax,8), %xmm0
        addpd   524304(%rdi,%rax,8), %xmm1
        addpd   524320(%rdi,%rax,8), %xmm0
        addpd   524336(%rdi,%rax,8), %xmm1
        addpd   524352(%rdi,%rax,8), %xmm0
        addpd   524368(%rdi,%rax,8), %xmm1
        addpd   524384(%rdi,%rax,8), %xmm0
        addpd   524400(%rdi,%rax,8), %xmm1
.Ltmp2:
        .loc    1 17 26                 # example4.c:17:26

        addq    $16, %rax
        jne     .LBB0_1
# %bb.2:                                # %middle.block
        #DEBUG_VALUE: test:x <- %rdi
        #DEBUG_VALUE: test:a <- %rdi
.Ltmp3:
        .loc    1 18 7                  # example4.c:18:7

        addpd   %xmm0, %xmm1
        movapd  %xmm1, %xmm0
        movhlps %xmm0, %xmm0            # xmm0 = xmm0[1,1]

        addpd   %xmm1, %xmm0
```

3 Performance Impacts of Vectorization

We will now familiarize ourselves with what code does/does not vectorize, and discuss how to increase speedup from vectorization.

3.1 The Many Facets of a Data Parallel Loop

In `loop.c`, we have written a loop that performs elementwise an operation — by default, addition — between two arrays `A` and `B`, storing the result in array `C`. If you examine the code, you will see that our loop does no useful work (in the sense that `A` and `B` are not filled with any initial values). We are just using this loop to demonstrate concepts. Further, we have added an outer loop over `I` whose purpose is to eliminate measurement error in `gettime()`.

Let's see what speedup we get from vectorization. Run `make` and run `awsrun ./loop`. Record the elapsed execution time. Then run `make VECTORIZE=1` and run `awsrun ./loop` again. Record the vectorized elapsed execution time. The flag `-mavx2` tells `clang` to use advanced vector extensions with larger vector registers. Run `make VECTORIZE=1 AVX2=1` and run `awsrun ./loop` again. **Note that you must use the `awsrun` machines for this; you may otherwise get a message like `Illegal instruction (core dumped)`.** You can check whether or not a machine supports the AVX2 instructions by looking for `avx2` in the `flags` section of the output of `cat /proc/cpuinfo`. Record the vectorized elapsed execution time.

> **Write-up 6:** What speedup does the vectorized code achieve over the unvectorized code? What additional speedup does using `-mavx2` give? You may wish to run this experiment several times and take median elapsed times; you can report answers to the nearest 100% (e.g., 2×, 3×, etc). What can you infer about the bit width of the default vector registers on the `awsrun` machines? What about the bit width of the AVX2 vector registers? *Hint*: aside from speedup and the vectorization report, the most relevant information is that the data type for each array is `uint32_t`.

3.1.1 Flags to enable and debug vectorization

Vectorization is enabled by default, but can be explicitly turned on with the `-fvectorize` flag[1]. When vectorization is enabled, the `-Rpass=loop-vectorize` flag identifies loops that were successfully vectorized, and the `-Rpass-missed=loop-vectorize` flag identifies loops that failed vectorization and indicates if vectorization was specified (see `Makefile`). Further, you can add the flag `-Rpass-analysis=loop-vectorize` to identify the statements that caused vectorization to fail.

3.1.2 Debugging through assembly code inspection

Another way to see how code is vectorized is to look at the assembly output from the compiler. Run

```
$ make ASSEMBLE=1 VECTORIZE=1
```

[1] If you open `Makefile`, you will see we set up things in a slightly different way. We set `-O3` regardless of vectorization—because we want a fair comparison when the vectorization flag is enabled/disabled. We then *disable* vectorization for when `VECTORIZE=0` by setting the flag `-fno-vectorize`.

This will produce `loop.s`, which contains human-readable x86 assembly like `perf annotate -f` from Recitation 2. Note that the compilation may "fail" with `ASSEMBLE=1` because this flag tells `clang` to not produce `loop.o`.

Write-up 7: Compare the contents of `loop.s` when the `VECTORIZE` flag is set/not set. Which instruction (copy its text here) is responsible for the vector add operation? Which instruction (copy its text here) is responsible for the vector add operation when you additionally pass `AVX2=1`? You can find an x86 instruction manual on LMOD. Look for MMX and SSE2 instructions, which are vector operations. To make the assembly code more readable it may be a good idea to remove debug symbols from release builds by moving the `-g` and `-gdwarf-3` CFLAGS in your Makefile. It might also be a good idea to turn off loop unrolling with the `-fno-unroll-loops` flag while you study the assembly code.

3.1.3 Flavors of vector arithmetic

As discussed in lecture, the vector unit is built directly in hardware. To support more flavors of vector operations (e.g., vector subtract or multiply), additional hardware must be added for each operation.

Write-up 8: Use the `__OP__` macro to experiment with different operators in the data parallel loop. For some operations, you will get division by zero errors because we initialize array `B` to be full of zeros—fix this problem in any way you like. Do any versions of the loop not vectorize with `VECTORIZE=1 AVX2=1`? Study the assembly code for `<<` with just `VECTORIZE=1` and explain how it differs from the AVX2 version.

The results may surprise you. For example, compare the results for `*` and `<<` (shift). The problem is that shifting by a variable amount (`B[j]`) is not a supported vector instruction unless we pass `-mavx2`. Changing `B[j]` to a constant value should allow the code to be vectorizable again.

3.1.4 Packing smaller words into vectors

A big class of optimizations you will use in future projects is optimizing data type width for your application. Consider the arrays `A`, `B`, and `C` which have data type `uint32_t` (given by the `__TYPE__` macro). Changing the data type for each array has an impact in two places:

1. Memory requirements. A smaller data type per element leads to a smaller memory footprint per array.

2. Vector packing. A smaller data type allows more elements to be packed into a single vector register.

Let's experiment with the vector packing idea:

> **Write-up 9:** What is the new speedup for the vectorized code, over the unvectorized code, and for the AVX2 vectorized code, over the unvectorized code, when you change `__TYPE__` to `uint64_t`, `uint32_t`, `uint16_t` and `uint8_t`? For each experiment, set `__OP__` to `+` and do not change `N`.

In general, speedup should increase as data type size decreases. This is a fundamental advantage over unvectorized codes where for fixed `N`, the number of instructions needed to perform elementwise operations over an array of `N` elements is *mostly* independent of the data type width.[2]

3.1.5 To vectorize or not to vectorize

Performance potential from vectorization is also impacted by what operation you wish to perform. Of the operations that vectorize (Section 3.1.3), multiply (`*`) takes the most clock cycles per operation.

> **Write-up 10:** You already determined that `uint64_t` yields the least performance improvement for vectorized codes (Section 3.1.4). Test a vector multiplication (i.e., `__OP__` is `*`) using `uint64_t` arrays. What happens to the AVX2 vectorized code's speedup relative to the unvectorized code (also using `uint64_t` and `*`)? What about when you set the data type width to be smaller — say `uint8_t`?

> **Write-up 11:** Open up the `aws-perf-report` tool for the AVX2 vectorized multiply code using `uint64_t` (as you did in Recitation 2). Remember to first use the `awsrun perf record` tool to collect a performance report. Does the vector multiply take the most time? If not, where is time going instead? Now change `__OP__` back to `+`, rerun the experiment and inspect `aws-perf-report` again. How does the percentage of time taken by the AVX2 vector add instruction compare to the time spent on the AVX2 vector multiply instruction?

[2] We say "mostly" because depending on your processor's architecture, arrays with large data types (e.g., 64 bit and 128 bit) are processed in different ways. For example, you *can* use 128 bit data types using gcc and the type `__int128`. But since ALUs in the `awsrun` machines are only 64 bits wide, the compiler turns each 128 bit operation into several 64 bit operations.

You will see that where time goes changes dramatically when you change `*` to `+`. This is partly due to the data type width (`uint64_t`) and partly due to the `*` operation itself. In particular, the awsrun machine vector units only support 32×32 bit multiplication—wider data types are synthesized from smaller operations. If you experiment with smaller (`uint16_t` and below) data types, you should see that the assembly code for `*` and `+` look more similar

3.2 Vector Patterns

We will now explore some common vector code patterns. We also recommend `https://llvm.org/docs/Vectorizers.html` as a reference guide for when you are optimizing your projects.

3.2.1 Loops with Runtime Bounds

Up to this point, our data parallel loop has been simple for the compiler to handle because `N` was known beforehand and was a power of 2. What about when the loop bound is not known ahead of time?

Write-up 12: Get rid of the `#define N 1024` macro and redefine `N` as: `int N = atoi(argv[1]);` (at the beginning of `main()`). (Setting `N` through the command line ensures that the compiler will make no assumptions about it.) Rerun (with various choices of `N`) and compare the AVX2 vectorized, non-AVX2 vectorized, and unvectorized codes. Does the speedup change dramatically relative to the `N = 1024` case? Why?

Hint: If you look at `loop.s` when you apply this change, you will see the compiler adding termination case code to handle the final loop iterations (i.e., the iterations that do not align with the vector register width). Test this yourself: as you set `__TYPE__` to smaller data types, you should see that the amount of termination-related assembly code emitted by the compiler increases.

3.2.2 Striding

Another simplifying feature in our loop is that its ***stride*** (or step) equals 1. Stride corresponds to how big our steps through the array are; e.g., `j++`, `j+=2`, etc. The `awsrun` machine vector units have some hardware support to accelerate different strides.

For example,

```
424 for (j = 0; j < N; j+=2) {
425   C[j] = A[j] + B[j];
426 }
```

Write-up 13: Set `__TYPE__` to `uint32_t` and `__OP__` to `+`, and change your inner loop to be strided. Does `clang` vectorize the code? Why might it choose not to vectorize the code?

`clang` provides a `#pragma Clang loop` directive that can be used to control the optimization of loops, including vectorization. These are described at the following webpage: `http://Clang.llvm.org/docs/LanguageExtensions.html#extensions-for-loop-hint-optimizations`

Write-up 14: Use the `#vectorize` pragma described in the `clang` language extensions webpage above to make `clang` vectorize the strided loop. What is the speedup over non-vectorized code for non-AVX2 and AVX2 vectorization? What happens if you change the `vectorize_width` to 2? Play around with the `clang` loop pragmas and report the best you found (that vectorizes the loop). Did you get a speedup over the non-vectorized code?

Once again, inspecting the assembly code to see how striding is vectorized can be insightful.

3.2.3 Strip Mining

A very common operation is to combine elements in an array (somehow) into a single value. For instance, one might wish to sum up the elements in an array. Replace the data parallel inner loop with such a reduction:

```
427 for (j = 0; j < N; j++) {
428   total += A[j];
429 }
```

To ensure that `clang` vectorizes the inner loop rather than the outer loop, comment out the outer loop.

Write-up 15: This code vectorizes, but how does it vectorize? Turn on `ASSEMBLE=1`, look at the assembly dump, and explain what the compiler is doing.

As discussed in lecture, this reduction will only vectorize if the combination operation (`+`) is associative.

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 7

Homework 4: Reducer Hyperobjects

In this homework you will experiment with parallelizing in Cilk. You will learn how to detect and solve determinacy races in your multithreaded code using Cilk Sanitizer, and how to measure a program's parallelism using the Cilkscale profiler.

1 Getting started

[Note: This assignment makes use of AWS and/or Git features which may not be available to OCW users.]

Submitting your solutions

For each question we ask give short responses of 1–3 sentences. Paste program outputs where necessary.

Reporting performance numbers

To test the performance of parallel code, you should use **`awsrun8`**, a variant of `awsrun` that submits jobs to cloud machines with 8 cores. It's important to use `awsrun8` for all performance numbers involving parallel code.

2 Recitation: Parallelism and race detection using Cilk

Please answer the checkoff questions on your own and show a TA after you have completed all of the questions in this section.

2.1 Introduction to Cilk

The `cilk_spawn` and `cilk_sync` keywords

Compile the `fib` program in the `fib/` subdirectory. Then, time the execution for finding `fib(45)` using the `time` command:

```
$ awsrun8 time ./fib 45
```

You will get 3 different times as outputs, labeled `real`, `user`, and `sys`. The `real` time is wall time, which is what you see from a clock. The `user` time is the time a CPU spent in user mode. The `sys` time is the time a CPU spent in the kernel. The sum of `user` and `sys` is the actual CPU time of the command. Since the current `fib` is a serial program, you will find that wall time is slightly higher than CPU time.

Next, we want to parallelize the program to take advantage of the other 7 processors on the `awsrun` machines. You can do this by adding `cilk_spawn` in front of function calls that you want to execute in parallel. You also need to add `cilk_sync` to wait for all spawning tasks to finish. Lastly, you should include the Cilk header using

```
01 #include <cilk/cilk.h>
```

You can specify the number of Cilk workers that execute a program by setting the environment variable `CILK_NWORKERS`. You can specify this variable globally for all Cilk programs executed in your bash session by running:

```
$ export CILK_NWORKERS=8
```

You may also specify the number of Cilk workers to 8 for a single execution of a program by prepending `CILK_NWORKERS=8` before the command you wish to run. Setting the number of Cilk workers is especially useful when executing a Cilk program when using `awsrun8`. For example, you may execute

```
$ awsrun8 CILK_NWORKERS=4 ./fib 45
```

This command will execute `fib` on the AWS job queue machines using 4 Cilk workers. You can safely ignore errors about not finding the command to set `CILK_NWORKERS` with `awsrun8` such as the following:

```
[Warning] Cannot find path for command: CILK_NWORKERS=8
```

Checkoff Item 1: Parallelize `fib` using `cilk_spawn` and `cilk_sync`, and report the runtime when using 1, 4, and 8 Cilk workers.

You might find that the new version is not faster than the first one (in terms of `real` time). Furthermore, the CPU time is much higher than wall time, because the program uses multiple processors to run. Under what circumstances would the parallel version be slower? Try to fix your program using "coarsening" to get a parallel speedup.

Checkoff Item 2: Describe your approach to coarsening the program, and report your parallel runtime for the coarsened version of `fib` on 1, 4, and 8 Cilk workers.

The `cilk_for` keyword

The `transpose/` subdirectory contains the source code for `transpose`, an in-place matrix-transpose program. As you saw in lecture, you can replace the `for` loops with `cilk_for` loops to parallelize `transpose`.

Checkoff Item 3: Parallelize `transpose` using `cilk_for`, and report your runtime with input size 10000 for 1, 4, and 8 Cilk workers.

2.2 The Cilksan race detector

The Cilksan race detector allows you to check whether your Cilk program has a determinacy race. It provides detailed output that specifies the line numbers of two memory locations, often a read and a write, that were involved in the race. The Cilksan tool needs the `libsnappy` package, which you can install on your AWS instance with:

```
$ sudo apt-get install libsnappy-dev
```

Compile and run the `qsort-race` program in the `qsort-race/` subdirectory. This code was parallelized by naively adding `cilk_spawn` and `cilk_sync` to a serial quicksort program. This code has a race!

Before you run Cilksan, take a look at the quicksort code and see if you can identify the determinacy race. See if you can expose the race condition by running a few tests on `awsrun8` too. Don't be discouraged if you are unable to expose the race by running tests — but also do not allow yourself to be fooled! This code does, indeed, have a race. Like many subtle determinacy races, the one present in quicksort is difficult to identify without the use of tools.

We can use Cilksan to detect the race as follows:

```
$ make clean; make CILKSAN=1
```

We can expose this race on a fairly small input of 10 elements:

```
$ ./qsort-race 10 1
```

Checkoff Item 4: Use Cilksan to find this race. Then, fix the race, and use Cilksan to confirm that no more races exist. Report the line numbers in the code where the read/write race occurs, and give a brief description of what was happening to cause this race.

2.3 The Cilkscale scalability analyzer

The `qsort/` subdirectory contains `qsort`, another parallel quicksort program. This version of quicksort should not have any races, but you should of course verify this by using Cilksan.

You can use Cilkscale to analyze the scalability of this quicksort program. We have printed the scalability information at the end of `main` in `qsort.c`:

```
02 #ifdef CILKSCALE
03 print_total();
04 #endif
```

We can build `qsort` with Cilkscale as follows:

```
$ make CILKSCALE=1
```

Since Cilkscale uses timing measurements to compute parallelism, it is usually a good idea to run it on a quiesced machine — such as those provided in the AWS job queue. For this assignment, however, we recommend you run Cilkscale locally on your AWS instance instead of using `awsrun`.

Checkoff Item 5: Report the parallelism computed by Cilkscale on the quicksort program for a few different sized inputs.

Cilkscale's command-line output includes work and span measurements for the Cilk program in terms of empirically measured cycle counts, as well as parallelism measurements based on the measured work and span. For some programs, such as `fib`, there may be some variability in the reported parallelism numbers.

Checkoff: Explain your responses to the previous five Checkoff Items to a TA.

3 Homework: N queens problem and reducers

We introduced the N Queens Problem in Lecture 3: Bit Hacks. The problem is to place N queens on a $N \times N$ chessboard so that no queen attacks another (i.e., no two queens in any row, column, or diagonal). Review the slides to get a feel for how the backtracking procedure works.

For this homework, our recursive implementation in `queens.c` (only 12 lines of code!) looks for all possible solutions and appends them to a list. It works for $N = 8$, a standard 8×8 chessboard. We chose $N = 8$ for two reasons:

1. Tractable number of solutions. $N = 8$ only has 92 distinct solutions. $N = 16$, for example, has 14772512 distinct solutions.

2. Simple board representation. The 8×8 squares of the chessboard fit into the bits of a `uint64_t`.

Lecture 3 represented the board as 3 bit vectors `down`, `left`, and `right` of size N, $2N - 1$, and $2N - 1$, respectively. The implementation does better! It uses the same 3 bit vectors, but of size N, N, and N only. This result was invented by Tony Lezard and can be found in an email from 1991:

```
Path: gmdzi!unido!mcsun!uknet!slxsys!ibmpcug!mantis!tony
From: to...@mantis.co.uk (Tony Lezard)
Newsgroups: rec.puzzles
Subject: Re: 8 Queens (NO *SPOILER*)
Message-ID: <eeFmBB1w164w@mantis.co.uk>
Date: 18 Nov 91 18:47:49 GMT
References: <1991Nov16.033939.70781@cs.cmu.edu>
Organization: Mantis Consultants, Cambridge. UK.
Lines: 40
redm...@cs.cmu.edu (Redmond English) writes:
> I wrote a little C program a while ago, which after exhaustively testing
> every position with exactly one queen on each rank (taking about 1 minute
> on an 8MHz 68000 machine), came up with 92 solutions.
Erk. Exhaustive testing? Count me out on that. The following program,
written in ANSI C, uses a backtracking algorithm and gives the correct
answer instantaneously:

#include <stdio.h>
void try(int, int, int);
static int count = 0;

void main() {
   try(0,0,0);
   printf("There are \%d solutions.\n", count);
}

void try(int row, int left, int right) {
   int poss, place;
```

```
      if (row == 0xFF) ++count;
      else {
         poss = ~(row|left|right) & 0xFF;
         while (poss != 0) {
            place = poss & -poss;
            try(row|place, (left|place)<<1, (right|place)>>1);
            poss &= ~place;
         }
      }
   }
```

```
   ObPuzzle: Generalize the above to n queens.
   --
   Tony Lezard IS to...@mantis.co.uk OR tony\%man...@uknet.ac.uk
   OR EVEN ar...@phx.cam.ac.uk if all else fails. Great! Kept my .sig down to two
   lines!
```

Write-up 1: (Bonus) How does the code snippet above work? Why do you only need N bits for each vector?

Our `queens` implementation uses three bit vectors, where 0s represent available squares and 1s represent unavailable squares. Therefore, the `down`, `left`, and `right` start as all 0s and get filled with 1s as the queens are placed.

3.1 It's a race, but no one is winning.

Compile and run `./queens`. Record the serial execution time.

Let's see if we can get any speedups from parallelization. In the `queens` function, add `cilk_spawn` and `cilk_sync` keywords to parallelize the recursive calls to `queens`. Compile and run `./queens` with `wsrun 8`.

When running a Cilk program, you may specify the number of worker threads you'd like to use by setting the environment variable `CILK_NWORKERS`. Specifying the number of Cilk workers can be done by adding `export CILK_NWORKERS=W` (where W is an integer greater than 0) in the command line before `./queens`. Most of the time, you won't have to explicitly set this variable because it defaults to the number of cores on the system. A good place to find Cilk-related reference information is cilk.mit.edu.

Write-up 2: What happened when you ran `./queens`? Did parallelization do what you expected?

Something went wrong. Let's use the Cilksan tool to detect the problem. Cilksan will take too long to finish with the current build, so comment out the loop which runs `run_queens I` times in the `main` function. **Remember to uncomment the loop before recording performance numbers for the remainder of the homework.**

Now compile with Cilksan. The `clang` option `-fsanitize=cilk` triggers compilation with Cilksan, and you can build with it automatically by running `make CILKSAN=1`. The resulting binary will run with Cilksan code built-in.

Write-up 3: Was Cilksan able to detect the problem? Describe the race condition briefly in words, and report the relevant line numbers for the read and write involved in the race.

3.2 Fixing the race

Remove `cilk_spawn` and `cilk_sync` keywords from the `queens` function. Before re-parallelizing our code, we want to make any accesses to the list of solutions, `board_list`, thread-safe.

Here's one strategy that doesn't require mutual exclusion locks: instead of passing in `board_list` to the recursive calls to `queens`, we can create a bunch of temporary `BoardList` objects and pass in a different one to each recursive thread. When the threads synchronize, all of the temporary lists will have been filled, and we can concatenate them all together with the original `board_list`.

In `queens.c`, implement

```
05 void merge_lists(BoardList* list1, BoardList* list2);
```

The function should merge `list2` into `list1` and then reset `list2` to be empty. For example, say `list1` has 3 nodes and `list2` has 5 nodes. After a call to `merge_lists(list1, list2)`, `list1` should have 8 nodes and `list2` should be empty. We'll use this call to concatenate several lists together. Remember to handle cases where one or both of the lists are empty and to update all 3 fields of `BoardList`.

Write-up 4: What is the most efficient way to concatenate two singly linked lists? What is the asymptotic running time of your `merge_lists` function?

Now, implement the strategy of passing in temporary lists to the recursive calls and merging them at the end. Compile and run `./queens` to make sure your serial code is still correct. If it is, re-parallelize the code with `cilk_spawn` and `cilk_sync`. Run Cilksan to verify that there are no races. Then record the parallel execution time.

> **Write-up 5:** Briefly explain your implementation. How does the parallel code perform compared to the serial code? Try to explain any difference in performance.

Recall that, in Homework 2, you coarsened the recursion of merge sort. Coarsen the recursion of `queens` so that if it hits a base case, then it uses the original implementation. Record the new parallel execution time. **Remember to uncomment the loop before recording performance numbers for the remainder of the homework.**

> **Write-up 6:** What is the base case you used? How does the parallel code with coarsening perform compared to the parallel code without? Try to explain any difference in performance. How does it perform compared to the serial code?

3.3 Cilk reducers

Background

Cilk provides a unique programming construct called a ***reducer hyperobject***, or ***reducer***. Conceptually, a reducer is a variable that can be safely used by multiple Cilk strands running in parallel. In a parallel execution of a program, the Cilk runtime maintains multiple views of a reducer. The runtime eliminates the possibility of a race by ensuring that each view can be accessed by only one Cilk strand at a time. After all parallel strands have been synchronized (i.e., after a `cilk_sync`), Cilk guarantees that all views of a reducer have been merged together into a single view. In Cilk, reducers are based on "monoids," and thus Cilk guarantees that a parallel execution of a program updates a reducer and merge views in a way that is equivalent to a serial execution of the same program.

More precisely, a ***monoid*** is an algebraic structure on a set of elements T with an associative binary operator $\oplus$ and an identity element e. As a simple example, for an integer sum monoid, T is $\mathbf{Z}$ (the set of integers), $\oplus$ is $+$ (integer addition), and the identity is $e = 0$.

The serial execution of a Cilk program typically performs a sequence of updates to a reducer X, which correspond to a sequence of $\oplus$ operations with elements x_i drawn from T. For example, consider a program with a reducer X with initial value x_0, and suppose that a serial execution of the program performs a sequence of updates $x_1, x_2, \ldots, x_7$ to a reducer X. The serial execution combines these updates one at a time, i.e., it produces a final value for X of

$$(((((((x_0 \oplus x_1) \oplus x_2) \oplus x_3) \oplus x_4) \oplus x_5) \oplus x_6) \oplus x_7) \; . \tag{1}$$

Because $\oplus$ is associative and has an identity element e, however, other ways to combine these updates also produce the same final result. For example, one could also combine updates as follows:

$$((((x_0 \oplus x_1) \oplus x_2) \oplus x_3) \oplus x_4) \oplus (((e \oplus x_5) \oplus x_6) \oplus x_7) \ , \tag{2}$$

and still produce the same result. The original serial execution order in Equation (1) uses a single ***view*** of X, updates occur sequentially. In contrast, the execution represented by Equation (2) uses two views, one for value $(x_0 \oplus x_1 \oplus x_2 \oplus x_3 \oplus x_4)$, and one for the value $(x_5 \oplus x_6 \oplus x_7)$. Having two views allows two processors to update X in parallel without races.

More generally, a parallel execution may create more reducer views and combine them in more complicated but equivalent ways. For example, Equation (3) uses three views, and Equation (4) uses five views.

$$(((x_0 \oplus x_1) \oplus x_2) \oplus ((e \oplus x_3) \oplus x_4)) \quad \oplus \quad (((e \oplus x_5) \oplus x_6) \oplus x_7) \tag{3}$$

$$((x_0 \oplus ((e \oplus x_1) \oplus x_2)) \oplus ((e \oplus x_3) \oplus x_4)) \quad \oplus \quad (((e \oplus x_5) \oplus (e \oplus x_6)) \oplus x_7) \tag{4}$$

In summary, reducers exhibit several attractive properties:

- Multiple strands can access a reducer without races.
- Reducers are shared without the need for mutual exclusion locks.
- Reducers can be used without significantly restructuring existing code.
- Defined and used correctly, reducers retain serial semantics. The result of a Cilk program that uses reducers is the same as the serial version, and the result does not depend on the number of processors or how the work is scheduled.
- Reducers are implemented efficiently, incurring little or no runtime overhead for synchronization. The cost of accessing a reducer is a hash-table lookup.

Implementing your own reducer

Cilk comes with a library of predefined reducers which support many commonly used monoids.

Here's an example usage of the + reducer:

```
#include <cilk/reducer_opadd.h>

// ...
  CILK_C_REDUCER_OPADD(r, double, 0);
  CILK_C_REGISTER_REDUCER(r);
  cilk_for(int i = 0; i != n; ++i) {
    REDUCER_VIEW(r) += A[i];
  }
  printf("The sum of the elements of A is %f\n", REDUCER_VIEW(r));
  CILK_C_UNREGISTER_REDUCER(r);
```

Since the list of boards `queens()` generates is not ordered, we can use a reducer to assemble it.

Write-up 7: How can a `BoardListReducer` help you to avoid creating the temporary lists in Section 3.2? Define the monoid: What type of objects will the reducer operate on? What is the associative binary operator? What is the identity object?

Unfortunately, Cilk did not come with a `BoardListReducer`, so let's implement our own in 6 easy steps:

1. Include the Cilk reducer header in your code:

```
#include <cilk/reducer.h>
```

2. Define the behavior of the reducer in 3 functions:

```
// Evaluates *left = *left OPERATOR *right.
void board_list_reduce(void* key, void* left, void* right) { ... }

// Sets *value to the the identity value.
void board_list_identity(void* key, void* value) { ... }

// Destroys any dynamically allocated memory. Hint: delete_nodes.
void board_list_destroy(void* key, void* value) { ... }
```

   Remember to cast the `void*` arguments to `BoardList*` before using them. You don't have to worry about the `key` argument (although you might see why it's there, since we previously mentioned a hash-table lookup).

3. Instantiate the reducer type:

```
25 typedef CILK_C_DECLARE_REDUCER(BoardList) BoardListReducer;
```

4. Initialize the reducer for use (it's easiest to put it in global scope):

```
26 BoardListReducer X = CILK_C_INIT_REDUCER(BoardList,              // type
27   board_list_reduce, board_list_identity, board_list_destroy,   // functions
28   (BoardList) { .head = NULL, .tail = NULL, .size = 0 });       // initial value
```

5. You must register and unregister the reducer before and after use. After the declaration of the reducer, but before the first use, insert `CILK_C_REGISTER_REDUCER(X)` to register. When the reducer is no longer needed, insert `CILK_C_UNREGISTER_REDUCER(X)` to unregister.
6. Use `REDUCER_VIEW(X)` to access the value. In this case, it will return a `BoardList`.
7. Once all parallel execution has completed, use `X.value` to query the final value of the reducer.

The full details are at:

`http://software.intel.com/en-us/articles/using-an-intel-cilk-plus-reducer-in-c`

You can find a good tutorial with a working example at:

`https://www.cilkplus.org/docs/doxygen/include-dir/page_reducers_in_c.html`

Write-up 8: Use a reducer to parallelize `queens`. Record the parallel execution time using `awsrun8 ./queens`. Verify that the answers you're getting are consistent with the serial code from before. **Note: Cilksan may report races if you do not explicitly call `__cilksan_disable_checking()` and `__cilksan_enable_checking()` before and after accesses to the reducer object.** How does the parallel code with reducers perform compared to the parallel code without?

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 9

Homework 5: Theory of Performance Engineering

Introduction

The focus of this problem set is on the theoretical side of the material taught in class and as well serves as a warm-up exercise for the quiz. This problem set should be done *individually*. Please treat it like a take-home quiz. Do not discuss problems with classmates, and clearly cite any external resources you use (e.g., books, published papers, wikipedia).

As preparation for the homework and quiz, read Chapter 27 of Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein (CLRS).

Please submit a write-up of your answers on the class site.

1 Greedy scheduling

Write-up 1: Solve Exercise 27.1-3 on Page 791 in Chapter 27 of CLRS.

Ben Bitdiddle measures his deterministic program on 4 and 64 processors on an ideal parallel computer using a greedy scheduler. He obtains running times of $T_4 = 100$ seconds and $T_{64} = 10$ seconds.

In the previous problem, we proved that any greedy scheduler schedules a computation with work T_1 and span T_∞ in time $T_P \leq (T_1 - T_\infty)/P + T_\infty$ on a P-processor ideal parallel computer. We also showed in lecture that $T_P \geq T_1/P$ (Work Law) and $T_P \geq T_\infty$ (Span Law). Based on these formulas, please answer the following questions.

Write-up 2:

1. What is the lowest possible value for the parallelism of the program?
2. What is the highest possible value for the parallelism of the program?

Ben Bitdiddle measures the running time of his deterministic parallel program scheduled using a greedy scheduler on an ideal parallel computer with 4, 10, and 64 processors. Ben obtains the following running times:

$$T_4 : 80$$

$$T_{10} : 42$$

$$T_{64} : 9$$

Write-up 3: Argue that Ben messed up at least one of his measurements.

Write-up 4: Solve Exercise 27.1-5 on Page 791 in Chapter 27 of CLRS.

2 Multithreaded matrix multiplication

Write-up 5: Solve Exercise 27.2-5 on Page 796 in Chapter 27 of CLRS.

Write-up 6: Solve Exercise 27.2-6 on Page 797 in Chapter 27 of CLRS.

3 Multithreading reductions and prefix computations

Write-up 7: Solve Problem 27-4 on page 807 in Chapter 27 of CLRS.

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 10

Homework 6: Custom Memory Allocators

[Note: This assignment makes use of AWS and/or Git features which may not be available to OCW users.]

1 Introduction

Project 3 requires you to examine the complex real-world problem of high-performance memory management. You will implement a serial memory allocator that implements the `malloc()`, `free()`, and `realloc()` functions (the C memory management API). In this homework, you will implement different versions of such an allocator.

Then, you will explore extensions to the memory management API for cases in which a custom memory allocator is useful. In particular, you will complete implementations for a "wrapped allocator," a "packed allocator," a "fixed aligned allocator," and a "smart allocator."

You should start installing OpenTuner immediately; it will take about 10 minutes to install, and starting it before you look at the code will let you have it ready once you need it. Please ensure that you are logged into your Amazon VM.

```
$ sudo apt-get install python-setuptools

$ sudo easy_install pip

$ sudo ./install_opentuner.sh
```

More detailed instructions are in the `README`. You can open up another `ssh` connection to your VM to work with while OpenTuner is being installed.

Performance

Unlike in previous assignments, you will be evaluating your custom allocators in terms of space utilization and throughput.

- ***Space utilization*** is the peak ratio between the aggregate amount of currently allocated memory (M) (i.e., allocated via your custom `malloc()` and not yet freed via your custom `free()`) and the size of the heap (H) used by your allocator. The optimal ratio is, of course, 1. The space utilization U is calculated as follows:

$$U = \max\{M, 40\text{KB}\} / \max\{H, 40\text{KB}\}$$

- ***Throughput*** is the average number of operations completed per second.

To summarize these two performance metrics for your allocator, we define a ***performance index*** P to be a weighted sum of the space utilization and throughput:

$$P = wU + (1 - w)\min\{1, T/T_{libc}\}$$

where U is your space utilization, T is your throughput, and T_{libc} is the estimated throughput of `libc`'s `malloc()` on your system on the default traces.

Running the code

Your allocator implementations will be tested on ***traces***, which are text files that encode a series of calls to `malloc()` and `free()`. You can use the provided `mdriver` program to test and evaluate your custom allocator on a given trace. Here is an example of how to compile and run the driver program on a particular trace:

```
$ make clean mdriver; awsrun ./mdriver -g -v -B -f traces/trace_c0_v0
```

You can also run the driver on all traces in the `traces` directory as follows:

```
$ awsrun ./mdriver -g -v -B
```

The `mdriver` program accepts the following command-line arguments:

- `-t <tracedir>`: Look for the default trace files in directory `<tracedir>` instead of the default directory (`./traces`).
- `-f <tracefile>`: Use one particular trace file for testing instead of the default set of trace files.
- `-h`: Print a summary of the command line arguments.
- `-l`: Run and measure `libc`'s `malloc()` in addition to the custom `malloc()` implementation.
- `-g`: Generate summary info for the autograder.

- `-v`: Verbose output. Print a performance breakdown for each trace file in a compact table.
- `-V`: More verbose output. Prints additional diagnostic information as each trace file is processed. Useful during debugging for determining which trace file is causing you to fail.
- `-B`: Use the custom "simple allocator."
- `-W`: Use the custom "wrapped allocator."
- `-P`: Use the custom "packed allocator."
- `-F`: Use the custom "fixed aligned allocator."
- `-S`: Use the custom "smart allocator."

2 Code layout

This homework uses code that resembles the code that you will use for Project 3. Let's review the functions whose implementations you will complete to implement your custom allocators, as well as some methods that you will use in those implementations.

Heap memory allocator interface

Your storage allocators will implement different versions of `init()`, `malloc()`, and `free()` using various allocation strategies. These functions are described below and (among other functions) are declared in `allocator_interface.h`. The specific versions of the functions to implement and modify are specified in each question.

- `int init(void);`

  Before calling the corresponding `malloc()` or `free()`, the application program (i.e., the trace-driven driver program that you will use to evaluate your implementation) calls `init()`. You may use this function to perform any necessary initialization, such as allocating the initial heap area. The return value should be -1 if there was a problem in performing the initialization and 0 if everything went smoothly. The specific versions you will encounter in this homework are

  ```
  int simple_init(void);
  int wrapped_init(void);
  int packed_init(void);
  int fixed_aligned_init(void);
  int smart_init(void);
  ```

- `void *malloc(size_t size);`

  This call must return a pointer to a contiguous block of newly allocated memory which is at least `size` bytes long. This entire block must lie within the heap region and must not overlap

any other currently allocated chunk. The pointers returned by `malloc()` must always be aligned to 8-byte boundaries; you'll notice that the `libc` implementation of `malloc()` does the same. If the requested size is zero or an error occurs and the requested block cannot be allocated, a `NULL` pointer must be returned. The specific versions you will encounter in this homework are

```
void *simple_malloc(size_t size);
void *wrapped_malloc(size_t size);
void *packed_malloc(size_t size);
void *fixed_aligned_malloc(size_t size);
void *smart_malloc(size_t size);
```

- `void free(void *ptr);`

  This call notifies your storage allocator that a currently allocated block of memory should be deallocated. The argument must be a pointer previously returned by `malloc()` and not previously freed. You are not required to detect or handle either of these error cases. However, you should handle freeing a `NULL` pointer – it is defined to have no effect. The specific versions you will encounter in this homework are

```
void simple_free(void *ptr);
void wrapped_free(void *ptr);
void packed_free(void *ptr);
void fixed_aligned_free(void *ptr);
void smart_free(void *ptr);
```

All of this behavior matches the semantics of the corresponding `libc` routines. Type `man malloc` at the shell to see additional documentation, if you're curious.

The provided memory allocator in `simple_allocator.c` is very fast. On `simple_malloc()`, it increases the heap size and returns the newly allocated memory, while on `simple_free()`, it does nothing. Compile and run it using `mdriver` on `rec_traces/trace_c0_v0`. Unsurprisingly, the reference allocator has nearly 0% space utilization (because it doesn't reuse freed memory) and 100% throughput.

Support routines

The code in `memlib.c` simulates the memory system for your dynamic memory allocators. You can invoke the following functions in `memlib.c`:

- `void* mem_sbrk(int incr);`

  Expands the heap by `incr` bytes, where `incr` is a positive non-zero integer and returns a generic pointer to the first byte of the newly allocated heap area. The semantics are identical to the Unix `sbrk()` function, except that `mem_sbrk()` accepts only a positive non-zero integer argument.

- `void* mem_heap_lo(void);`

  Returns a generic pointer to the first byte in the heap.

- `void* mem_heap_hi(void);`

  Returns a generic pointer to the last byte in the heap.

- `size_t mem_heapsize(void);`

  Returns the current size of the heap in bytes.

- `size_t mem_pagesize(void);`

  Returns the system page size in bytes (4 KB on Linux systems).

In addition to these functions, there are several macros defined in `allocator_interface.h` to help you implement your custom allocators.

3 Fixed-size blocks

To improve the space utilization, let's implement the fixed-size allocation strategy from Lecture 11: Storage Allocation. In this strategy, all blocks are allocated with the same size. The allocator uses a free list to track the freed blocks. The free list can be implemented as a singly linked list, with the `next` pointers stored inside the freed blocks.

For this first part, assume that the fixed block size that we use is 1024 bytes. Add the following lines to the top of `simple_allocator.c`:

```
#ifndef BLOCK_SIZE
#define BLOCK_SIZE 1024  // default value
#endif
```

You should use the variable `BLOCK_SIZE` in your code rather than hardcoding 1024 as you will be modifying `BLOCK_SIZE` in later parts. Notice that next to each block, the provided memory allocator stores the size of the block because it is needed for memory reallocation. Even though you are not using any reallocation functionality in this homework (that will be in Project 3!), you can choose whether or not you still want to store the size next to each block (which requires allocating slightly extra memory).

Checkoff Item 1: Implement the fixed-size block allocation strategy in `simple_allocator.c` by modifying `simple_init()`, `simple_malloc()`, and `simple_free()` as necessary. Report that the space utilization (and score) increases to over 99% when run on `rec_traces/trace_c0_v0`. *Hint:* Make a `struct` for the nodes of the free list, and remember to initialize the head in `simple_init()`. Use the `-B` flag when you run `mdriver`.

4 Autotuning

Unlike `rec_traces/trace_c0_v0`, trace `rec_traces/trace_c1_v0` only requests memory of a fixed size `BLOCK_SIZE = 4096`. Your allocator should support all allocation sizes less than or equal to `BLOCK_SIZE`. Add the following lines to the top of the `simple_malloc()` function in `simple_allocator.c`:

```
if (size > BLOCK_SIZE)
  return NULL;
else  // size <= BLOCK_SIZE
  size = BLOCK_SIZE;
```

The `simple_malloc()` function is now able to handle all sizes less than or equal to `BLOCK_SIZE`, and it returns an error (as a `NULL` pointer) for sizes greater than `BLOCK_SIZE`. (You may argue that this error checking shouldn't be necessary in `simple_malloc()`: someone else should check that it's broken if it returns a block of size `BLOCK_SIZE` when a size greater than `BLOCK_SIZE` was requested. We agree, and that will be your job in Project 3 when you fill in `validator.h`.) Confirm that there is an error when you recompile and run on `rec_traces/trace_c1_v0`.

```
$ make clean mdriver; ./mdriver -g -v -B -f rec_traces/trace_c1_v0
```

Next, let's manually override `BLOCK_SIZE` and confirm that there is no longer an error.

```
$ make clean mdriver PARAMS="-D BLOCK_SIZE=4096"

$ ./mdriver -g -v -B -f rec_traces/trace_c1_v0
```

`BLOCK_SIZE` is an example of a tunable parameter of the code, but how do we determine the best value for `BLOCK_SIZE`? In this case, we could easily determine the value by manually inspecting the traces, but in general, it can be very difficult to tune these parameters by hand, particularly when there are multiple different parameters (as you will see in Project 3). This is where autotuning is useful.

OpenTuner is a autotuning tool that, by running an optimization, automatically finds the best values for the parameters that you tell it about. You will use OpenTuner next to see if you can automatically determine the appropriate values for `BLOCK_SIZE` on different traces.

Checkoff Item 2: Use OpenTuner to find the best value of `BLOCK_SIZE` for `rec_traces/trace_c0_v0` and `rec_traces/trace_c1_v0`.

1. Add `BLOCK_SIZE` as a power-of-two parameter in `opentuner_params.py`, varying it from 2^5 to 2^{15}.

2. Run the OpenTuner script (which takes 1–2 minutes):

   ```
   $ ./opentuner_run.py --test-limit=300 --no-dups --display-frequency=20 \
   --trace-file=<trace-file>
   ```

   How did OpenTuner know that `BLOCK_SIZE` should be `1024` and `4096`, respectively? Is it just really good at reading traces, or is the value of `BLOCK_SIZE` somehow affecting the value of the optimization's objective function?

3. Add a target `autotune` to your Makefile so that you can run OpenTuner by running

   ```
   $ make clean autotune TRACE_FILE=<trace_dir>/<tracefile>
   ```

   Hint: Your Makefile target needs to run the `opentuner_run.py` script with the necessary flags shown above, as well as the trace file passed in on the command line.

Checkoff

Commit your changes to your local repository, then verify your work using `verifier.py` and check your code quality by running `clint.py`. If these scripts pass, show your work to a TA or UTA to complete the checkoff for the recitation.

5 Cache-friendly allocation

Aligning objects on a cache-line boundary limits the number of cache lines needed to access an object. For randomly accessed objects, ensuring that an access uses the fewest cache lines possible is especially important.

In implementing the allocators in this section, assume that objects never need to be freed with `wrapped_free()`. You can evaluate your allocators empirically on `traces/trace_c0_nofree`. Any existing allocator can be used to ensure that objects start at a cache-line boundary with sufficient padding.

Write-up 1: In `wrapped_allocator.c`, wrap the call to `unaligned_malloc()` inside of `wrapped_malloc()` to ensure that each object starts at a cache-line boundary. The wrapper code cannot assume anything about the state of `unaligned_malloc()`. The macros in `allocator_interface.h` may prove useful. From analyzing your allocator's code and looking at its utilization when run through `mdriver`, argue about how much memory is wasted for aligned allocations. Use the `-W` flag when you run `mdriver`.

Although each object will need fewer cache lines, inefficient cache utilization can lead to *more* cache loads overall.

Now consider allocating memory in a more cache-friendly way that allows for a more compact packing. In other words, you can pack multiple objects into a single cache-line. Even though some of your objects may no longer be cache-aligned, don't forget that objects still need to be 8-byte aligned. The allocator in `packed_allocator.c` is targeting a single-threaded workload and aims to ensure that each object spans a minimal number of cache lines. The provided memory allocator stores the size of a block next to that block. Let's keep this block header and think about where we need to store it in relation to the pointer returned to the caller and what we might need to store in it.

Write-up 2: Implement `packed_malloc()` in `packed_allocator.c`. Where did you allocate the block header? Report the utilization and performance scores. Based on analyzing your code and running it through `mdriver`, how much memory is wasted overall by your aligned allocator? Use the `-P` flag when you run `mdriver`.

6 Allocator overheads

Write-up 3: Do we need to allocate the header in `packed_allocator.c` if, instead of the classic `free(p)` interface, programmers were in charge of passing the original size as in `free(p, size)`?

Write-up 4: Assume that we have only one object size for our next allocator: 64 bytes. Do we need to allocate a size header at all? Fill in the `fixed_aligned_init()`, `fixed_aligned_malloc()`, and `fixed_aligned_free()` methods in `fixed_aligned_allocator.c` to implement a cache-aligned fixed-size allocator. (In particular, make sure you implement a free list this time.) Show the utilization and performance of your allocator. Use the `-F` flag when you run `mdriver`.

Now assume that we need to support large (64-byte) and small (32-byte) object sizes. We still wish for these objects to be cache-aligned, that is, at least one endpoint of each object must lie on a cache-line boundary. Although we could use large allocations for all objects, this internal fragmentation can double the memory requirements. Can we support allocation and deallocation of two sizes with **zero** space overhead?

Let's implement these ideas in `smart_allocator.c`. Suppose that you can change the interface for your allocator such that any pointer returned by `smart_malloc()` needs to be accessed through the `SMART_PTR()` macro.

Write-up 5: Where and how can you store the size of each allocation? Following this assumption, the `smart_free()` implementation in `smart_allocator.c` uses the `SMART_PTR()` and `IS_SMALL()` macros to operate on a given pointer. Implement the `SMART_PTR()` and `IS_SMALL()` macro definitions in `allocator_interface.h`.

Write-up 6: Using a single allocator for both different sizes might still suffer from external fragmentation. If the small and large objects need to be cache-aligned, then we might need to waste space to allocate a 64 byte object on a cache line boundary. What would you do with the "wasted space"? Implement `alloc_aligned()` and `smart_malloc()` in `smart_allocator.c`, and show your utilization. (Use the `-S` flag with `mdriver`.)

Write-up 7: *(No implementation required.)* What can you do when you need to allocate a 32-byte object when you have free-list entries of size 64? What would happen once you have run out of space and keep breaking up large objects, but never coalesce small, adjacent objects into large ones? What can you do when you have run out of space and need to allocate a 64-byte object when you have free list entries of size 32?

Write-up 8: *(No implementation required.)* Allocating space has to be done in contiguous regions of memory. Is it possible to coalesce two 32-byte objects into a 64-byte object if the 32-byte objects are not adjacent in memory? How would you implement this coalescing if possible?

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 12

Homework 7: Dynamic-Analysis Tools

[Note: This assignment makes use of AWS and/or Git features which may not be available to OCW users.]

1 Introduction

Key to understanding and improving the behavior of any system is ***visibility*** — the ability to know what is going on inside the system. For application and system software, ***compiler-inserted program instrumentation*** (or simply ***compiler instrumentation***) — where the compiler inserts special code into the program-under-test to monitor its execution — has emerged as a popular way for programmers to gain visibility into how their programs are operating. In your previous assignments, you have already availed yourself of a variety of dynamic-analysis tools, such as the Cilksan race detector, Valgrind, AddressSanitizer, Cachegrind, and perf. These tools generally operate as ***shadow computations*** — executing behind the scenes while the program-under-test runs. In this homework, you will investigate how to build your own dynamic-analysis tool using compiler instrumentation.

Traditionally, to write a dynamic-analysis tool that uses compiler instrumentation, a tool writer must modify the source code of the compiler. But modifying a compiler requires substantial mastery of the compiler's source code that is beyond the scope of this course. In this homework, you will use ***CSI***, a prototype framework supported by the Tapir/LLVM compiler for writing dynamic-analysis tools.

The CSI framework

CSI aims to simplify many aspects of writing and using tools that employ compiler instrumentation by providing comprehensive static instrumentation and removing the need for tool writers to interact directly with the compiler codebase. Specifically, CSI provides a simple application program interface (API) consisting of functions, called ***hooks***, which are automatically inserted

throughout the compiled code of the program-under-test. The CSI API exposes generic instrumentation points throughout the program-under-test, thereby hiding the complexity of the compiler's codebase from tool writers.

Using CSI, tool writers insert their own instrumentation into the program-under-test by writing a library, called a ***CSI-tool***, that defines the semantics of relevant hooks. All unimplemented hooks in the CSI-tool default to null behavior. A tool user uses the CSI framework to link the compiled CSI-tool with the program-under-test to produce a ***tool-instrumented executable (TIX)***. When the TIX executes, the program-under-test runs normally, except that whenever a hook is invoked, the tool performs its shadow computation. In order to support a wide variety of tools, CSI inserts hooks to instrument many events that a tool might care about, for instance, before and after a memory access, function entry and function exit, beginning and end of a basic block, etc.

Figure 1 presents CSI-cov, an example tool written using the CSI framework. CSI-cov reports code-coverage information for an execution of the TIX of a program-under-test. As the TIX executes, CSI-cov records when each basic block runs. When the TIX terminates, CSI-cov reports how many times each block was executed, including its location in the source code. The code for CSI-cov, shown in Figure 1 in its entirety, is both short and simple.[1] In just 40 lines, the CSI framework allows this useful compiler-based tool to be implemented as a simple C library.

CSI targets program points that are visible in LLVM's intermediate representation (IR). In particular, CSI instruments six ***categories*** of ***IR objects***: memory loads, memory stores, basic blocks, function entry points, function exits, and call instructions.

The design of the CSI API is centered around two main considerations: (1) to allow for simple library-based implementations of a wide-variety of dynamic-analysis tools, and (2) to enable tools to run quickly by exploiting standard compiler optimizations and analyses for precision and efficiency. To these ends, the CSI API provides four key features: hooks, flat and compact CSI ID spaces, forensic tables, and properties. We overview each of the features in turn.

Flat CSI ID spaces. To each IR object, CSI assigns a ***CSI ID***, an integer identifier for the IR object that is unique within its category. At runtime, these ID's are passed to hooks to identify the particular IR object or objects being instrumented. CSI assigns these ID's in such a way as to maintain a flat and compact ID space for each IR-object category, even when ***translation units*** — components of a program that are separately compiled — are dynamically loaded. These flat and compact ID's greatly simplify tool design by allowing a tool to use simple arrays, rather than hash tables, to store information for each IR object. Such arrays are simpler to maintain than hash tables, especially in multithreaded programs, and they often perform more efficiently than hash tables.

Hooks. CSI defines two kinds of hooks: initialization hooks and IR-object hooks. The initialization hooks allow tools to perform actions immediately before `main` is invoked and when a translation unit is loaded. The IR-object hooks allow tools to perform actions before and/or after an IR object is executed at runtime. The compiler automatically elides any hooks left undefined by the tool writer.

[1] This code as written assumes that the program-under-test executes serially. Additional care must be taken for allocating, initializing, and accessing `block_executed` in order for CSI-cov to handle a multithreaded program-under-test. Thread-safety is an issue that tool writers must address explicitly.

```c
#include <stdio.h>
#include <stdlib.h>
#include <csi.h>

long *block_executed = NULL;
csi_id_t num_basic_blocks = 0;

void report() {
  csi_id_t num_basic_blocks_executed = 0;
  fprintf(stderr, "CSI-cov report:\n");
  for (csi_id_t i = 0; i < num_basic_blocks; i++) {
    if (block_executed[i] > 0)
      num_basic_blocks_executed++;
    const source_loc_t *source_loc = __csi_get_bb_source_loc(i);
    if (NULL != source_loc)
      fprintf(stderr, "%s:%d:%d executed %d times\n",
              source_loc->filename, source_loc->line_number,
              source_loc->column_number, block_executed[i]);
  }
  fprintf(stderr, "Total: %ld of %ld basic blocks executed\n",
          num_basic_blocks_executed, num_basic_blocks);
  free(block_executed);
}

void __csi_init() {
  atexit(report);
}

void __csi_unit_init(const char * const name,
                     const instrumentation_counts_t counts) {
  block_executed = (long *)realloc(block_executed,
                                   (num_basic_blocks + counts.num_bb) *
                                   sizeof(long));
  memset(block_executed + num_basic_blocks, 0, counts.num_bb * sizeof(long));
  num_basic_blocks += counts.num_bb;
}

void __csi_bb_entry(const csi_id_t bb_id, const bb_prop_t prop) {
  block_executed[bb_id]++;
}
```

Figure 1: A serial version of the CSI-cov code-coverage tool, which reports the number of times every basic block in a TIX is executed.

Forensic tables. In addition to hooks for IR objects, the CSI API defines ***forensic*** tables which store static information created by the compiler. A CSI-tool can access these tables at runtime through abstract accessor functions. The prototype CSI implementation in Tapir/LLVM provides forensic tables that associate IR objects with locations in the source code, as well as tables that specify the number of LLVM IR instructions in each basic block.

The code of CSI-cov in Figure 1 demonstrates that CSI ID's, hooks, and forensic tables can be used in a tool. CSI-cov maintains a table `block_executed`, defined in line 5, in which each basic block in the TIX has a unique slot indexed by the basic block's CSI ID. The number of basic blocks is stored in the variable `num_basic_blocks`, defined in line 6.

The initialization hook `__csi_init` is invoked before `main` of the program-under-test is called. This hook, defined in lines 25–27, registers the function `report`, defined in lines 8–23, to run when the program terminates. The `report` function prints out a report summarizing the results of the tool's analysis. For each basic block, `report` retrieves its source location from CSI's forensic tables using the accessor function `__csi_get_bb_source_loc` in line 14. It prints the file name, the initial source line, and the number of times the basic block has been executed in lines 16–18. Finally, `report` prints the total number of basic blocks executed and the total number of basic blocks in the program in lines 20–21.

The hook `__csi_unit_init` is invoked whenever a new translation unit is loaded, either statically or dynamically. The hook is called with the name of the translation unit and a structure that provides the number of IR objects in each IR-object category that the translation unit contains. CSI-cov defines this hook in lines 29–36 to reallocate the array `block_executed` to incorporate the new basic blocks in lines 31–33, after which it updates the number `num_basic_blocks` of basic blocks in line 35.

The hook `__csi_bb_entry`, defined in lines 38–40, is called every time a basic block is executed. The hook has two arguments: the CSI ID of the basic block and a "property," which CSI-cov ignores. At runtime, CSI-cov indexes `block_executed` and increments the counter for that basic block.

Properties. The one key feature of CSI that the CSI-cov example does not illustrate is the use of properties. For each IR object, CSI computes a ***property***, which is a compile-time constant that encodes the results of standard compiler analyses. The property for a memory operation, for example, specifies that the location is guaranteed to be on the stack and has a particular alignment. The CSI framework uses properties to optimize the inserted instrumentation. In particular, if a CSI-tool branches based on the value of a property, CSI can constant-fold the test and optimize the instrumentation accordingly.

As an example of how a CSI tool might use properties, imagine that a tool writer builds a race detector capable of detecting races on shared variables. If she were using conventional compiler instrumentation, she could avoid instrumenting locations that could not possibly be involved in a race, such as a variable declared `const` or a variable on the stack whose address does not escape the frame. In CSI, the property argument to an IR-object hook can give her access to similar specific compile-time information concerning the memory operation being instrumented.

Figure 2 shows a snippet of the code that the writer of a race-detection tool might produce. In defining the hook `__csi_before_load`, the code safely skips the instrumentation of a load if it satisfies certain criteria: it accesses a `const` value (line 45), it accesses a memory location

```
41 void __csi_before_load(const csi_id_t load_id,
42                         const void *addr,
43                         const int32_t num_bytes,
44                         const load_prop_t prop) {
45   if (prop.is_const ||
46       prop.is_not_shared ||
47       prop.is_read_before_write_in_bb)
48     return;
49   check_race_on_load(addr, num_bytes);
50 }
```

Figure 2: An example memory-load hook for a race detector that uses properties to avoid unnecessary instrumentation.

whose address is not captured and is thus guaranteed not to be shared (line 46), or it reads a memory address that is written to within the same basic block without any intervening function calls (line 47). Because these conditions are known at compile time, the CSI framework can evaluate the condition when it inserts this load hook before each memory load. As a result, the CSI framework will preclude the compiler from inserting runtime calls to `__csi_before_load` before loads that satisfy these conditions. The function is analyzed at compile time and executed at runtime only when necessary. If a given property holds at link time and a tool branches depending on that condition, CSI can constant-fold the test and optimize the instrumentation.

Architecture of CSI

At first glance, CSI's brute-force method of inserting hooks at every salient location in the program-under-test seems replete with overheads. For example, the CSI instrumentation of a memory operation involves calls to two hooks, one before the operation and one after. If a tool does not use these hooks, the cost of the function calls can contribute significantly and unnecessarily to runtime overhead.

The CSI prototype overcomes overheads through the use of modern compiler technology, specifically, through the use of ***link-time optimization (LTO)***. Conceptually, LTO allows compiler optimizations to run when the program's compiled units are statically linked together, thereby allowing the compiler to perform optimizations between these separate units. Because all unimplemented hooks in a CSI-tool default to a null behavior, LTO can remove calls to these hooks as dead code. Furthermore, LTO can can constant-fold the test and optimize the instrumentation based on property values and perform a variety of optimizations on the instrumentation code itself.

From the point of view of the tool writer, CSI works as follows. The tool writer defines relevant hooks for her CSI-tool, and then she statically links her tool with a ***null tool***, which provides a no-op implementation of every hook. Linking these two tools thus produces a ***null-default CSI-tool***, where the functions defined in the tool writer's CSI-tool override the corresponding functions in the null tool, but for hooks not defined in the tool writer's CSI-tool, the null-hook definition defaults. When a tool user statically links a null-default CSI-tool with a

program-under-test, LTO automatically elides calls to null hooks during its optimization pass.

2 Recitation: Analyzing function executions

In recitation, you will use CSI to develop tools to analyze the execution of functions in a serial program. You will first develop a simple tool to measure ***function coverage***, that is, to count how many times each function in the program executes. Time permitting, you will then extend the function-coverage tool to compute a ***profile*** for the program-under-test, which specifies how much of the total execution time of the program-under-test is spent in each function. For this recitation assignment, refer to Section 4 for documentation on the CSI API.

The file `func-analysis.c` provides a boiler-plate for the function-analysis CSI-tool. The check-off items in this homework will walk you through completing the implementation of this CSI-tool.

Checkoff Item 1: Add global variables to `func-analysis.c` for recording the number of times each function in the program executes. Add code to the `__csi_init` and `__csi_unit_init` instrumentation hooks to initialize these global variables.

Checkoff Item 2: Add a function `report` to `func-analysis.c` for outputting the results of the analysis. In particular, `report` should use CSI forensic tables to output the name of each function in the program and how many times that function was executed. Register the `report` function to run at program termination by adding the statement `atexit(report);` to `__csi_init`.

Checkoff Item 3: Which CSI IR-object hooks should be implemented to observe when each function is executed? Implement those IR-object hooks in `func-analysis.c` to record the number of times each function executes.

Checkoff Item 4: Compile the function-analysis CSI-tool and null tool and instrument `fib_serial` with the function-analysis tool as follows:

```
51 $ make CSI=1 FUNCANALYSIS=1 fib_serial
```

Finally, run the instrumented `fib_serial` executable.

Checkoff Item 5: Once you have confirmed that function-coverage CSI-tool is working, use it to instrument and analyze `bzip2`. We have already modified the `Makefile` for `bzip2` to optionally make use of a specified CSI-tool. Hence, you can instrument `bzip2` with your function-coverage CSI-tool and test the executable as follows:

```
52 $ cd bzip2-1.0.6
53 $ make test CSI=../func-analysis.o
54 $ ./bzip2 -z -9 -c ../bzip2-1.0.6.tar > /dev/null
```

Checkoff: Demonstrate to a TA that the function-coverage tool correctly counts the number of times each function in `bzip2` is executed.

Checkoff: *(Time permitting.)* Modify the function-coverage tool to profile each function, that is, to record the amount of the running time of the program is spent within each function. Try out the tool on `fib` and on `bzip2`. Make sure the tool doesn't over-count the time spent in recursive functions! Demonstrate to a TA that the profiling tool works.

3 Homework: Linear-regression analysis

In this homework, you will write some simple CSI-tools to perform ***linear-regression analysis***. Linear regression is a linear approach to modelling the relationship between a scalar ***dependent variable*** (in this case runtime) and one or more ***independent variables***. The idea behind linear-regression analysis is to develop a simple, predictive linear model to describe the performance of a program. Such a ***performance model*** provides guidance for how one can tune the performance of a program. There are many possible independent variables in a program that can affect it's running time, including:

- Lines of code. Some lines of code are more expensive than others.
- C language primitives. Some primitives are more expensive. Vectorizable instructions can make things cheaper.
- Machine instructions. Some instructions are more expensive than others e.g. square root compared to increment.
- Machine instructions weighted by their cost. Some instructions vary in cost e.g. a load with a cache hit vs. a cache miss.
- Cache Misses. TLB misses.

In this homework, we will guess that we can model the time T to run a program as following equation:

$$T = a \cdot I + b \cdot C \ ,$$

where I is the number of instructions, C is the number of cache misses, and a and b are constant values. Linear-regression analysis involves solving for the constants a and b in this model in two steps. First, one accumulates a variety of measurements of the program's running-time and corresponding instruction and cache-miss counts. Given those measurements, one can use a program such as gnuplot or Excel to perform a linear-regression fit of this model to the measurements, thereby solving for the constants a and b in the model. The constant a describes the cost of an instruction, and the constant b describes the cost of a cache miss in the program.

This homework uses linear-regression analysis to measure the cost of two short-running program events. First, you will measure the cost of a function call. Second, you will measure the cost of a `cilk_spawn`. You will study to programs, `fib_serial` and `fib_cilk`, to perform the linear-regression analyses in this homework. You can compile these programs without instrumentation as follows:

```
55 $ make fib_serial fib_cilk
```

As with the recitation assignment, refer to the documentation of the CSI API in Section 4 when writing CSI-tools for this homework.

Measuring the cost of a function call

To estimate the cost of a single function call, let us consider the serial recursive exponential-time `fib` function in the file `fib_serial.c`. This execution of this `fib` function is comprised of function calls and integer arithmetic and logic. Hence, we can model the execution time T of this code using the following linear-regression model:

$$T = a \cdot I + b \cdot F \,, \tag{1}$$

where I is the number of (non-function-call) instructions, F is the number of function calls, and a and b are constants. Using this model, we can estimate the cost of a function call by performing a linear-regression analysis to solve for the constants a and b.

To perform this analysis, we need to observe several measurements of the running-time T of different executions of `fib` that each perform different, uncorrelated numbers of instructions I and function calls F. To control the number of instructions and function calls that an execution of `fib` performs, the program `fib_serial.c` takes two parameters: the nth Fibonacci number to compute, and a ***base-case size***, which must be at least 2. The program then computes the nth Fibonacci number using an exponential-time recursive routine until `fib` is recursively called on an input value n smaller than the base-case size. At that point, the routine uses a simple loop to compute the requested Fibonacci number.

Write-up 1: Implement a CSI-tool in `linreg.c` to count the number of LLVM IR instructions and function calls that a program execution performs. You can get the number of LLVM IR instructions in a basic block using `__csi_get_bb_sizeinfo`. Use the `non-empty-size` value for the LLVM IR instruction count for a basic block. Submit your CSI-tool for this write-up.

Note: The prototype CSI implementation currently contains a bug that affects its instrumentation on simple recursive functions, such as `fib`. To get around this bug, declare all counter variables in your tool as `volatile`.

One issue with compiler instrumentation is that the instrumentation code can perturb the execution of the program-under-test. In particular, a program instrumented with the CSI-tool from Write-up 1 will perform more operations than the original uninstrumented program. As a result, the instrumented program might run more slowly than the uninstrumented program.

Write-up 2: Describe your strategy for collecting the necessary measurements to perform this linear-regression analysis using timers (e.g., calls to `clock_gettime`) and the CSI-tool from Write-up 1. Your strategy should address the following questions:

- What inputs will you use to run the program such that the counts of instructions and function calls are not highly correlated?
- How will you measure the running time of the program? What timers will you use? How many times will you run the program on each input? How will you aggregate your measurements to ensure you have reliable timings (e.g., using min, mean, or median)?
- How will you accommodate the fact that program instrumentation can perturb the running time of the program?

Now use your CSI-tool from Write-ups 1 and 2 to perform the linear-regression analysis.

Write-up 3: Following the strategy you outlined in Write-up 2, use timers and the CSI-tool from Write-up 1 to collect the running-time measurements and instruction and function-call counts for the `fib_serial.c` program. You can compile the linear-regression CSI-tool and instrument `fib_serial` with it as follows:

```
56 $ make CSI=1 LINREG=1 fib_serial
```

Use these measurements to perform the linear-regression analysis to solve for constants a and b in (1). You can use any data-analysis program you prefer (e.g., gnuplot or Excel) to perform the linear-regression fit on the collected measurements. What values does your analysis find for the constants a and b? How correlated are the counts of instructions and function calls in your measurements?

Measuring the cost of a Cilk spawn

Now you will use linear-regression analysis to measure the cost of a `cilk_spawn` statement. To perform this analysis, consider the Cilk program in `fib_cilk.c`, which computes Fibonacci numbers in exponential time. The serial running time of this program T_1 can be modeled using the following linear-regression model:

$$T_1 = a \cdot I + b \cdot S, \tag{2}$$

where I is the number of instructions (including serial function calls), S is the number of `cilk_spawn`'s, and a and b are constants. Using this model, one can estimate the cost of a `cilk_spawn` by performing a linear-regression analysis to solve for the constants a and b. For this analysis, we are primarily interested in solving for the constant b.

To solve (2) for the constant b, the `fib_cilk.c` program can be run using different numbers of `cilk_spawn` statements to compute a particular Fibonacci number. The `fib` routine in `fib_cilk.c` takes a base-case size along with the value n specifying which Fibonacci number to compute. When `fib` is called on a value of n larger than the base-case size, then the first recursive call to `fib` is spawned using a `cilk_spawn`. Otherwise this recursive call is called serially.

Write-up 4: Modify the CSI-tool you implemented in Write-up 1 to additionally count the number of `cilk_spawns` executed by a program-under-test. The prototype CSI framework in the Tapir/LLVM compiler inserts the following hook at the point in LLVM's IR corresponding to a `cilk_spawn`:

```
void __csi_detach(const csi_id_t detach_id);
```

To simplify the design of your tool, you can assume that you will run the program-under-test with `CILK_NWORKERS=1`. Submit your CSI-tool for this write-up.

Write-up 5: Mirroring the steps outlined in Write-ups 2 and 3, use the CSI-tool developed in Write-up 4 to perform the linear-regression analysis to estimate the cost of a `cilk_spawn`. You can compile the linear-regression CSI-tool and instrument `fib_cilk` with it as follows:

```
03 $ make CSI=1 LINREG=1 fib_cilk
```

When taking measurements for this analysis, run the `fib_cilk` executable using `CILK_NWORKERS=1`. Describe your strategy for taking the necessary timing measurements and instruction and `cilk_spawn` counts. What values does your analysis compute for the constant b? How correlated are the counts of instructions and spawns in your

```
typedef int64_t csi_id_t;
// Value representing unknown CSI ID
#define UNKNOWN_CSI_ID ((csi_id_t) -1)

typedef struct {
  csi_id_t num_func;
  csi_id_t num_func_exit;
  csi_id_t num_callsite;
  csi_id_t num_bb;
  csi_id_t num_load;
  csi_id_t num_store;
} instrumentation_counts_t;

// Hooks to be defined by tool writer
void __csi_init();
void __csi_unit_init(const char * const file_name,
            const instrumentation_counts_t counts);
```

Figure 3: CSI hooks for initialization.

measurements? Based on the linear-regression analysis from Write-up 3, what is the cost of a `cilk_spawn` in terms of function calls?

4 Reference: The CSI API

This section overviews the CSI API implemented in the Tapir/LLVM compiler. The CSI API consists of four key features: CSI ID spaces, hooks, properties, and forensic tables. The API employs flat and compact CSI ID spaces to identify IR objects. The API provides hooks that are invoked for IR objects, as well as hooks for initializing a CSI tool. The API exports results of standard compiler analysis through property parameters, allowing the tool writer to avoid unnecessary instrumentation. Forensic tables associate IR objects with locations in the source code and relate IR objects to each other. This section describes each of these four key features.

CSI ID's

CSI identifies six categories of IR objects: function entries, function exits, basic blocks, call sites, loads, and stores. Each IR object is assigned a unique number, called a ***CSI ID***, within each category. The CSI ID's are consecutively numbered from 0 up to 1 less than the number of IR objects in the category. A CSI ID has type `csi_id_t`.

```
21 void __csi_func_entry(const csi_id_t func_id,
22                        const func_prop_t prop);
23 void __csi_func_exit(const csi_id_t func_exit_id,
24                       const csi_id_t func_id,
25                       const func_exit_prop_t prop);
26
27 void __csi_before_call(const csi_id_t callsite_id,
28                         const csi_id_t func_id,
29                         const call_prop_t prop);
30 void __csi_after_call(const csi_id_t callsite_id,
31                        const csi_id_t func_id,
32                        const call_prop_t prop);
```

Figure 4: CSI hooks for functions.

```
33 void __csi_bb_entry(const csi_id_t bb_id,
34                      const bb_prop_t prop);
35 void __csi_bb_exit(const csi_id_t bb_id,
36                     const bb_prop_t prop);
```

Figure 5: CSI hooks for basic blocks.

Tool initialization

CSI provides two initialization hooks, shown in Figure 3. The global initialization hook `__csi_init` executes exactly once immediately before the program-under-test invokes the `main` function and before it initializes global variables. The unit-initialization hook `__csi_unit_init` is executed once whenever a translation unit is loaded into the TIX, whether statically or dynamically.

IR object hooks

CSI provides hooks for the each of the six categories of IR objects it identifies. To provide flexibility to tool writers, CSI generally inserts a hook both just before and just after each IR object. This flexibility allows, for example, a memory location's value to be queried before a store, after a store, or both.

Functions are instrumented on entry and exit. The hook `__csi_func_entry` is invoked at the beginning of every instrumented function instance after the function has been entered and initialized but before any user code has run — in LLVM terminology, at the first insertion point of the entry block of the function. The `func_id` parameter identifies the function being entered or exited. Correspondingly, the hook `__csi_func_exit` is invoked just before the function returns (normally).[2] Its parameters include both a function ID `func_id` and a function-exit ID `func_exit_id`, which allows tool writers to distinguish the potentially multiple exit points from a function.

[2]We have not yet defined the API for exceptions.

```
37 void __csi_before_load(const csi_id_t  load_id,
38                         const void *addr,
39                         const int32_t num_bytes,
40                         const load_prop_t prop);
41 void __csi_after_load(const csi_id_t load_id,
42                       const void *addr,
43                       const int32_t num_bytes,
44                       const load_prop_t prop);
45
46 void __csi_before_store(const csi_id_t store_id,
47                         const void *addr,
48                         const int32_t num_bytes,
49                         const store_prop_t prop);
50 void __csi_after_store(const csi_id_t store_id,
51                        const void *addr,
52                        const int32_t num_bytes,
53                        const store_prop_t prop);
```

Figure 6: CSI hooks for loads and stores.

The `__csi_before_call` and `__csi_after_call` hooks instrument call sites. The `callsite_id` parameter identifies the call site, and the `func_id` parameter identifies the ***callee*** — the function being called. It is not possible at compile time to identify with certainty the callee of a call site if the callee is called indirectly via a function pointer or if the callee is not instrumented by CSI. In these cases, the `func_id` argument is set to `UNKNOWN_CSI_ID`, a macro defined by the CSI library, as shown in Figure 3.

Figure 5 shows the two CSI hooks for basic blocks. The hook `__csi_bb_entry` is called when control enters a basic block, and `__csi_bb_exit` is called just before control leaves the basic block. The `bb_id` parameter identifies the basic block being entered or exited.

Figure 6 shows the four CSI hooks for memory operations. The hooks `__csi_before_load` and `__csi_after_load` are called before and after memory loads, respectively, and likewise, `__csi_before_store` and `__csi_after_store` are called before and after memory stores. The argument `addr` is the location in memory, and `num_bytes` is the number of bytes loaded or stored.

Properties

Every IR-object hook contains a ***property*** parameter `prop`: a bit-field struct that encodes static information for optimizing instrumentation. CSI defines a distinct bit-field struct type for every IR-object category: `func_prop_t`, `func_exit_prop_t`, `call_prop_t`, `bb_prop_t`, `load_prop_t`, `store_prop_t`. In the prototype CSI implementation in Tapir/LLVM, Figures 7 and 8 presents the property struct types that have been implemented in the existing CSI prototype. As an example, consider the `load_prop_t` type for loads, which is shown in Figure 8. The property encodes the alignment of the load (`alignment`) — an integer k indicating that the address must be a multiple of k — whether the load is volatile (`is_volatile`), whether the loaded location is guaranteed not to be shared (`is_not_shared`), etc.

```
54 typedef struct {
55   // The call is indirect.
56   unsigned may_spawn : 1;
57   // Pad struct to 64 total bits.
58   uint64_t _unused : 63;
59 } func_prop_t;
60
61 typedef struct {
62   // The function might spawn.
63   unsigned may_spawn : 1;
64   // Pad struct to 64 total bits.
65   uint64_t _unused : 63;
66 } func_exit_prop_t;
67
68 typedef struct {
69   // The function might spawn.
70   unsigned is_indirect : 1;
71   // Pad struct to 64 total bits.
72   uint64_t _unused : 63;
73 } call_prop_t;
```

Figure 7: Definition of the CSI property for function entry points, function exits, and call sites.

Forensic tables

In addition to properties, CSI passes information known at compile time through its ***forensic tables***. The forensic tables map CSI ID's to static information associated with the instrumented IR objects. CSI encapsulates the tables with a set of accessor functions, which allow the tool writer to map IR objects to their source-code locations, such as line numbers and containing file name, and to get instruction counts associated with each basic block. Figure 9 presents the accessors for the forensic tables implemented in the CSI prototype implementation in Tapir/LLVM.

```
74 typedef struct {
75   // The alignment of the load.
76   unsigned alignment : 8;
77   // The loaded address is in a vtable.
78   unsigned is_vtable_access : 1;
79   // The loaded address points to constant data.
80   unsigned is_constant : 1;
81   // The loaded address is on the stack.
82   unsigned is_on_stack : 1;
83   // The loaded address cannot be captured.
84   unsigned may_be_captured : 1;
85   // The loaded address is read before it is written in the same basic block.
86   unsigned is_read_before_write_in_bb : 1;
87   // Pad struct to 64 total bits.
88   uint64_t _unused : 51;
89 } load_prop_t;
90
91 typedef struct {
92   // The alignment of the store.
93   unsigned alignment : 8;
94   // The stored address is in a vtable.
95   unsigned is_vtable_access : 1;
96   // The stored address points to constant data.
97   unsigned is_constant : 1;
98   // The stored address is on the stack.
99   unsigned is_on_stack : 1;
100  // The stored address cannot be captured.
101  unsigned may_be_captured : 1;
102  // Pad struct to 64 total bits.
103  uint64_t _unused : 52;
104 } store_prop_t;
```

Figure 8: Definition of the CSI property for memory loads and memory stores.

```
105 // Structure for the source-location information of an instrumented IR
106 // object.
107 typedef struct {
108   // The name of the instrumented IR object, such as the name of the
109   // function.  This field is NULL if no name is found.
110   char * name;
111   int32_t line_number;
112   int32_t column_number;
113   char * filename;
114 } source_loc_t;
115
116 // Accessors for various CSI forensic tables that associate IR objects
117 // with locations in the source code.  These accessors return NULL
118 // when given an invalid ID.
119 const source_loc_t *__csi_get_func_source_loc(const csi_id_t func_id);
120 const source_loc_t *__csi_get_func_exit_source_loc(const csi_id_t func_exit_id);
121 const source_loc_t *__csi_get_bb_source_loc(const csi_id_t bb_id);
122 const source_loc_t *__csi_get_call_source_loc(const csi_id_t call_id);
123 const source_loc_t *__csi_get_load_source_loc(const csi_id_t load_id);
124 const source_loc_t *__csi_get_store_source_loc(const csi_id_t store_id);
125
126 // Structure for the count of LLVM IR instructions within a basic
127 // block.
128 typedef struct {
129   // Total LLVM IR instructions.
130   int32_t full_ir_size;
131   // Number of LLVM IR instructions that are implemented with at least
132   // one instruction in a machine architecture, e.g., no debug
133   // intrinsic function calls or phi instructions.
134   int32_t non_empty_size;
135 } sizeinfo_t;
136
137 const sizeinfo_t *__csi_get_bb_sizeinfo(const csi_id_t bb_id);
```

Figure 9: CSI accessor functions for reading forensic tables for source location information and for reading the number of LLVM IR instructions in each basic block.

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 13

Homework 8: Cache-Oblivious Algorithms

Then, answer the writeup questions in this handout and submit an *individual* writeup. See the following paper for more information on cache-oblivious algorithms: `https://dl.acm.org/citation.cfm?id=2071383`.

For this homework, assume that all matrices are stored in row-major layout.

1 Cache Complexity of Matrix Multiplication

During Lecture 14 we discussed the cache complexity of matrix multiplication of dimension n, with tall cache assumption of size $\mathcal{M}$ and cache line size $\mathcal{B}$. For the naive approach, there were two cases: 1) If $n > \mathcal{M}/\mathcal{B}$, then $\Theta(n^3)$ cache misses occur, and 2) if $\mathcal{M}^{1/2} < n < \mathcal{M}/\mathcal{B}$, then $\Theta(n^3/\mathcal{B})$ cache misses occur. For the blocking approach, with block size $s < \mathcal{M}^{1/2}$, $\Theta(n^3/\mathcal{B}\mathcal{M}^{1/2})$ cache misses occur. The cache-oblivious approach achieves the same complexity as the blocking approach without the need of the voodoo parameter s.

Checkoff Item 1: Assume we want to multiply two rectangular matrices: $m \times n$ with $n \times r$. Given the same tall cache assumption, please analyze the complexity for one of the following four cases: the two cases for the naive approach ($n > \mathcal{M}/\mathcal{B}$ and $\mathcal{M}/r < n < \mathcal{M}/\mathcal{B}$), the block approach, and the cache-oblivious approach. You may pick whichever case you want to analyze.

2 Tableau Construction

Consider the tableau-construction problem from Lecture 8. The problem involves filling an $N \times N$ tableau, where each entry of the tableau is calculated as a function of some of its neighbors. To be specific, the equation to fill an element of the tableau would take the form

$$A[i][j] = f(A[i-1][j-1], A[i][j-1], A[i-1][j])$$

where f is an arbitrary function.

2.1 Iterative Formulation

Consider the code snippet in Figure 1 below.

```
01 #define A(i, j) A[N + (i) - (j) - 1]
02
03 void tableau(double *A, size_t N) {
04   for (size_t i = 1; i < N; i++) {
05     for (size_t j = 1; j < N; j++) {
06       A(i, j) = f(A(i-1, j-1), A(i, j-1), A(i-1, j));
07     }
08   }
09 }
```

Figure 1: A simple, iterative loop for filling a tableau.

In this problem, we are only interested in computing the final value of the tableau, stored in `A(N-1,N-1)`, and hence we really only need $2N - 1$ amount of space during computation. Thus, the algorithm declares `A` as an array of size $2N - 1$.

The algorithm initializes the first row and first column of the tableau, and invokes the `tableau` function as shown in Figure 2.

```
10 for (size_t i = 0; i < N; i++) {
11   A(i, 0) = INIT_VAL;
12 }
13 for (size_t j = 0; j < N; j++) {
14   A(0, j) = INIT_VAL;
15 }
16 tableau(A, N);
17 res = A(N - 1, N - 1);
```

Figure 2: Initializing and calling the iterative `tableau` function.

Write-up 1: Explain why $2N - 1$ space is sufficient and how the `tableau` function utilizes the $2N - 1$ space.

Recall the tall cache assumption, which states that $\mathcal{B}^2 < \alpha\mathcal{M}$, where $\mathcal{B}$ is the size of the cache line, $\mathcal{M}$ is the size of the cache, and $\alpha \leq 1$ is a constant.

Write-up 2: Assuming that an optimal replacement strategy holds and that the cache is tall, give a tight upper bound on the cache complexity $Q(n)$ for each of the following cases using O notation, where $c \leq 1$ is a sufficiently small constant:

1. $n \geq c\mathcal{M}$
2. $n < c\mathcal{M}$

2.2 Recursive Formulation

Now consider the code snippet for a recursive tableau implementation, as shown in Figure 3. This

```
18 #define A(i, j) A[N + (i) - (j) - 1]
19
20 void recursive_tableau(double *A, size_t rbegin, size_t rend, size_t cbegin,
21                         size_t cend) {
22   if (rend-rbegin == 1 && cend-cbegin == 1) {
23     size_t i = rbegin, j = cbegin;
24     A(i, j) = f(A(i-1, j-1), A(i, j-1), A(i-1, j));
25   } else {
26     size_t rmid = rend-rbegin > 1 ? (rbegin + (rend-rbegin) / 2) : rend;
27     size_t cmid = cend-cbegin > 1 ? (cbegin + (cend-cbegin) / 2) : cend;
28     recursive_tableau(A, rbegin, rmid, cbegin, cmid);
29     if (cend > cmid)
30       recursive_tableau(A, rbegin, rmid, cmid, cend);
31     if (rend > rmid)
32       recursive_tableau(A, rmid, rend, cbegin, cmid);
33     if (rend > rmid && cend > cmid)
34       recursive_tableau(A, rmid, rend, cmid, cend);
35   }
36 }
```

Figure 3: A recursive implementation for filling in a tableau.

algorithm similarly uses only $2N - 1$ amount of space, initializes the array `A`, and invokes the `recursive_tableau` function as shown in Figure 4. This recursive algorithm divides the tableau into four quadrants to compute. As discussed in Lecture 8 (slide 88), after the first quadrant is done computing, we can then compute the second and third quadrants in parallel. Parallelizing this way gives us work as $\Theta(n^2)$ and span as $\Theta(n^{\lg 3})$ with parallelism as $\Theta(n^{2-\lg 3})$. We also discussed (slide 92) a more parallel construction that divides up the tableau 9 ways.

```
37 for (size_t i = 0; i < N; i++) {
38   A(i, 0) = INIT_VAL;
39 }
40 for (size_t j = 0; j < N; j++) {
41   A(0, j) = INIT_VAL;
42 }
43 if (N > 1) {
44   recursive_tableau(A, 1, N, 1, N);
45 }
46 res = A(N-1, N-1);
```

Figure 4: Initializing and calling the `recursive_tableau` function.

Write-up 3: Derive the general formula for work and span, assuming a k^2-way tableau construction (i.e., the tableau is divided up into k^2 pieces of size $n/k \times n/k$).

Write-up 4: Answer the following questions assuming that an optimal replacement strategy holds and that the cache is tall.

1. Show the recurrence relation for the cache complexity $Q(n)$ using the 4-way construction of the `recursive_tableau` function.
2. Draw the recursion tree and label the internal nodes and leaves with their cache complexity $Q(n)$. What's the height of the recursion tree?
3. How many leaves are in the recursion tree?
4. Using the recursion tree and the recurrence relation, derive a simplified expression for $Q(n)$.

Write-up 5: Answer the following question assuming that an optimal replacement strategy holds and that the cache is tall. Assuming a k^2-way tableau construction, show that if we are "unlucky," where a subpiece is just slightly above the cache size, then we have $Q(n) = \Theta(n^2 k/\mathcal{MB})$. Also show that if we are lucky and this situation does not arise, then we have $Q(n) = \Theta(n^2/\mathcal{MB})$.

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 14

Homework 9: Deterministic Execution

Please answer the questions in this handout and submit an *individual* writeup.

[Note: This assignment makes use of AWS and/or Git features which may not be available to OCW users.]

1 Introduction

We'll explore the power of deterministic execution, which may or may not help you debug your final project. Before we even get to parallel code, there will be plenty of bugs to root out in your serial code.

2 Environment dependencies

The most entertaining bugs to debug are the ones that depend on the environment. Why would a bug happen only on `awsrun` but not on your development machine? Why would code that works great for you, occasionally crash for your partner, and always for your TA?

Let's dig deeper into this simple program excerpt from `undef.c`.

```
01 main() {
02   int i;
03   printf("value of i=%d\n", i);
04   printf("address &i=%p\n", &i);
05 }
```

Figure 1: An undef.c excerpt

The `Makefile` contains a number of useful targets for this exercise. Whenever you use a target, you may find it useful to examine the `Makefile` in detail and understand why certain targets have the result they do.

Run it a few times using `make undef-compare` and compare the results. Are they the same? Why or why not? Now, let's fix these nondeterministic outputs line by line.

Whenever your program is using undefined state, it's often going to produce non-deterministic results. Undefined variables (or bits within one) are one such nondeterminism source. Fix the code to define the variable and rerun `make undef-compare` to make sure that it worked.

Next, we will look at nondeterminism in addresses. The higher order bits of a pointer are random due to Address Space Layout Randomization (ASLR) (look on Wikipedia for more details), which is an important security feature now in all modern operating systems. However, for testing how it impacts your address-dependent nondeterministic program, you can run your program without ASLR with the command `setarch x86_64 -R ./undef`. Run `make undef-noaslr` to run the program a few times without ASLR. Is this program deterministic now?

Even after considering these two factors, there are even more aspects that affect a nondeterministic program. Run `make undef-env` to run the command with differing environments. What is printed now? To see why, run `make whoami` and take a look at `user.c` :

```
06 int main(int argc, char *argv[], char* envp[]) {
07   char *username=genenv("USER");
08 }
```

Figure 2: An excerpt from user.c

You can see that the stack location is perturbed by the environment block, which affects the lower order bits of the stack variable *i* in `undef.c`.

Thus, we learned that program addresses are thus an important source of nondeterminism. These are a derivative of environmental, time, and randomness sources beyond our control.

3 Nondeterministic Hashtable

Now let's see a more complicated example in `hashtable.c`. It has a mostly working implementation of an open address hashtable with linear probing. This technique is a lot more cache-friendly compared to hashtables with linked lists.

Run `make hashtable1` a few times. It may work for you most of the time. How about `make hashtable10` or `make hashtable1000`?

> **Checkoff Item 1:** What do you need to make this program deterministic? Modify the Makefile target for `hashtable1000-good` so that all runs succeed.

To fix our bugs, we want the bugs to be reproducible.

Checkoff Item 2: Modify `hashtable1-bad` target so the program always fails. Show your modified Makefile target. You may find it useful to examine `hashtable.c` to see what system arguments it takes.

4 Replay Debugging

To fix serial code, you can use the GCC Record/Replay debugging facilities:

http://www.sourceware.org/gdb/wiki/ProcessRecord/Tutorial

```
(gdb) break main
(gdb) run
(gdb) record
(gdb) continue
(gdb) reverse-next
(gdb) reverse-next
(gdb) reverse-continue
```

If you don't like typing, you can just do

```
(gdb) b main
(gdb) r
(gdb) rec
(gdb) c
(gdb) rn
(gdb) rn
(gdb) rc
```

You may find it useful to watch some variables when debugging:

```
(gdb) set can-use-hw-watchpoints 0
(gdb) watch some_variable_name
```

You don't necessarily have to use good tools—if you have plenty of time, you can always figure out any bug by staring at the code long enough (static analysis by eyeballing). However, a much easier approach is to run your code with asserts, printf statements, or replay debugging techniques like gdb. Use any of the above techniques to find and fix the bug in `hashtable_insert`.

Checkoff Item 3: What was the bug? What is your fix? Rerun `make hashtable1000` to ensure that your fix always works.

Remember that this same bug may appear in other functions that have similar routines. This is an important reason why you should try not to write the same piece of code twice. An easy way to fix this is to refactor your code or use macros.

5 Hashlocking

Assuming that the serial code works, let's move on to the parallel version with `make hashtable-mt`. If you run `make hashtable-mt1000`, you may find some errors. We will need some synchronization here to fix the errors.

One possibility is to use a single hashtable lock, as in `hashtable_insert_locked`. However, this lock would be very highly contended. A good technique for reducing lock contention on a shared data structure is to split the lock into multiple locks. For example, instead of a single "tablelock", we consider having a separate lock for each section of the hashtable. A lock per hashtable line ("rowlock") would have too much memory and cache overhead, but a lock per hash ("hashlock") gives us the amount of overhead that we want.

Modify `hashtable_fill` to use `hashtable_insert_fair`, which is a rather basic implementation of a hashlock.

Write-up 1: Is the hashlock implementation in `hashlock.c` vulnerable to false sharing? Why or why not?

Write-up 2: What problem is the fairness solution in `hashtable_insert_fair` introducing? Explain or demonstrate the behavior you see.

6 Lockless Hashtable

Now, let's try to avoid using a lock altogether. On modern processors, the assembly instruction `CMPXCHG16B` supports the compare-and-swap operation for 16 bytes, and similar instructions exist for 4 bytes and 8 bytes. However, early 64-bit processors didn't support the 16 byte instruction. In this vein, we'll use the `CMPXCHG8B` in our implementation to only work with 64-bit words. This does mean that we can't atomically write a whole `entry_t`. Assume that `size` can be a positive integer only. The relevant definitions are in `common.h`.

Write-up 3: Use `InterlockedCompareExchange64` to implement your changes in `hashtable_insert_lockless`. Don't forget to modify `hashtable_fill` to use your new code. Run `make hashtable-mt1000` to ensure that it works. What changes are necessary in `hashtable_lookup` so you can use just a 8-byte compare-and-swap in `hashtable_insert_lockless`?

7 Deterministic Hashtable

Write-up 4: (No implementation required.) How would you implement a deterministic hashtable structure, such that the order of insertions does not change the final hashtable state?

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms

Performance Engineering of Software Systems
Massachusetts Institute of Technology 6.172
Prof. Charles E. Leiserson and Prof. Julian Shun Handout 16

Homework 10: Data Synchronization

Introduction

The focus of this problem set is the theoretical side of the material taught in class. This problem set focuses on data synchronization and comparing lock-based and lock-free FIFO queue implementations.

1 Data synchronization

Figures 1 and 2 present two implementations of a FIFO queue. Figure 1 is a lock-based implementation, and Figure 2 is a lock-free implementation. In both queue implementations, a pool of nodes is allocated in advance. A call to `new_node()` grabs a free node from the pool of nodes, and `free_node(node)` returns the node `node` to the pool. In the implementation of the lock-free queue, the compare-and-swap instruction `CAS(addr, old_val, new_val)` is an atomic instruction that has the following effect:

```
if (*addr == old_val) {
  *addr = new_val;
  return true;
}
return false;
```

For the questions below, assume that `CAS` can operate on the entire `pointer_t`, that the compiler cannot change the order of instructions, and that there are always enough free nodes in the pool to perform all enqueue operations. Assume also that the nodes in the queue do not cross cache lines, and thus all writes are atomic.

Read both implementations carefully. Before you start answering the questions, you may find it helpful to draw diagrams of an empty queue and a queue with a few nodes. Using these diagrams, try to understand how nodes are inserted and deleted from the queue in both implementations.

Note that the first node added in initialization of both the lock-based and lock-free version of the deque is a dummy value and never dequeued. It is only used to denote an empty deque.

2 Check-off Questions

1. What are constraints on `enqueue` and `dequeue` in the FIFO queue? You do not need to look at the code yet.

2. What is the advantage of using two locks over one lock?

3. In the style of comments of the lock-based FIFO queue code, add comments to the lock-free code (on paper), explaining what each line does. The comments should be short and precise (not more than 10 words each). We have provided you a copy of the code in Figure 2.

4. Explain how a new node is inserted into the lock-free queue. How many successful `CAS`es are needed per node? What happens if the `CAS` in line 96 fails? How far can the tail lag behind? Is the program correct without line 96?

5. Carefully look at the code for the lock-free dequeue operation and answer the following questions:

   (a) Line 104 checks what was already assigned in line 101. Why do we need line line 104?

   (b) In line 111 the value of the node is read before the head is updated in line 112. Why is this important? What can happen if we change the order of the lines?

   (c) What happens if the `CAS` in line 112 is unsuccessful?

6. Which implementation do you expect to run faster — the lock-based or the lock-free? Explain your answer in terms of cost of the synchronization primitives, contention, synchronization overhead, etc.

7. Show how to simplify the lock-based code if only one thread may enqueue nodes to the queue. Write the pseudocode and comment it. Explain in your own words why your solution is correct (i.e. any execution sequence keeps the FIFO ordering).

8. Show how to simplify the lock-free code if only one thread may dequeue nodes from the queue. Write the pseudocode and comment it. Explain in your own words why your solution is correct (i.e. any execution sequence keeps the FIFO ordering) and why it is non-blocking.

9. Explain how `count` is used to handle the ABA problem discussed in recitation.

```
11 struct node_t {
12   data_t value;
13   node_t* next;
14 };
15 struct queue_t {
16   node_t* head;
17   node_t* tail;
18   mutex_t h_lock;
19   mutex_t t_lock;
20 };
21
22 void initialize(queue_t* q, data_t value) {
23   node_t* node = new_node();  // Allocate a new node
24   node->value = value;
25   node->next = NULL;          // Make it the only node in the queue
26   q->head = node;             // Both head and tail point to it
27   q->tail = node;
28   q->h_lock = FREE;           // Locks are initially free
29   q->t_lock = FREE;
30 }
31
32 void enqueue(queue_t* q, data_t value) {
33   node_t* node = new_node();  // Allocate a new node
34   node->value = value;        // Copy enqueued value into node
35   node->next = NULL;          // Set next pointer of node to NULL
36   lock(&q->t_lock);           // Acquire t_lock to access tail
37   q->tail->next = node;        // Append node at the end of queue
38   q->tail = node;              // Swing tail to node
39   unlock(&q->t_lock);         // Release t_lock
40 }
41
42 bool dequeue(queue_t* q, data_t* pvalue) {
43   lock(&q->h_lock);           // Acquire h_lock to access head
44   node_t* node = q->head;     // Read head
45   new_head = node->next;      // Read next pointer
46   if (new_head == NULL) {     // Is queue empty?
47     unlock(&q->h_lock);       // Release h_lock before return
48     return false;             // Queue was empty
49   }
50
51   *pvalue = new_head->value;  // Queue not empty. Read value
52   q->head = new_head;         // Swing head to next node
53   unlock(&q->h_lock);         // Release h_lock
54   free_node(node);            // Free node
55   return true;                // Dequeue succeeded
56 }
```

Figure 1: C-like pseudocode declaring, initializing, and adding for lock-based FIFO queue.

```
57 struct pointer_t {
58   node_t* ptr;
59   unsigned int count;
60 };
61 struct node_t {
62   data_t value;
63   pointer_t next;
64 };
65 struct queue_t {
66   pointer_t head;
67   pointer_t tail;
68 };
69
70 void initialize(queue_t* q, data_t value) {
71   node_t* node = new_node();
72   node->value = value;
73   node->next.ptr = NULL;
74   q->head.ptr = node;
75   q->tail.ptr = node;
76 }
77
78 void enqueue(queue_t* q, data_t value) {
79   node_t* node = new_node();
80   node->value = value;
81   node->next.ptr = NULL;
82   pointer_t tail;
83   while (true) {
84     tail = q->tail;
85     pointer_t next = tail.ptr->next;
86     if (tail == q->tail) {
87       if (next.ptr == NULL) {
88         if (CAS(&tail.ptr->next, next, (struct pointer_t) {node, next.count + 1 })) {
89           break;
90         }
91       } else {
92         CAS(&q->tail, tail, (struct pointer_t) {next.ptr, tail.count + 1 });
93       }
94     }
95   }
96   CAS(&q->tail, tail, (struct pointer_t) { node, tail.count + 1 });
97 }
```

Figure 2: C-like pseudocode for declaring, initializing, and adding to a lock-free FIFO queue.

```
bool dequeue(queue_t* q, data_t* pvalue) {
  pointer_t head;
  while (true) {
    head = q->head;
    pointer_t tail = q->tail;
    pointer_t next = head.ptr->next;
    if (head == q->head) {
      if (head.ptr == tail.ptr) {
        if (next.ptr == NULL) {
          return false;
        }
        CAS(&q->tail, tail, (struct pointer_t) { next.ptr, tail.count + 1});
      } else {
        *pvalue = next.ptr->value;
        if (CAS(&q->head, head, (struct pointer_t) { next.ptr, head.count + 1})) {
          break;
        }
      }
    }
  }
  free_node(head.ptr);
  return true;
}
```

Figure 3: C-like pseudocode for dequeueing in a lock-free FIFO queue.

MIT OpenCourseWare
https://ocw.mit.edu

6.172 Performance Engineering of Software Systems

Fall 2018

For information about citing these materials or our Terms of Use, visit: https://ocw.mit.edu/terms